



GumBall6900

The index fund
that chooses itself

A plain-English whitepaper for a signal-directed onchain fund. Anyone can mine GBX from a fixed public schedule, stake it into non-transferable sGBX, and continuously direct where the fund's next dollar goes. Acquisitions clear against the open market. Redemption pays out real tokens, in kind. The deployed core is fixed: signals govern capital, code governs the rules.

Mine GBX. Signal acquisitions. Build shared backing. Redeem onchain.

by Heesho

1,000,000,000

GBX LIFETIME MINT
CEILING

98%

PUBLIC MINING CAPACITY

Zero

TEAM OR INVESTOR
ALLOCATION

Three

ONGOING MANAGEMENT
ACTIONS

READ THIS FIRST

This describes reviewed code, not a deployed system.

GumBall6900 is experimental software. It is **not deployed**, has had **no independent external audit**, and is **not authorized for user funds**. This paper explains what the production contracts do, what internal review found, and what remains unfinished. Nothing here is investment, legal, or tax advice, and nothing here is a promise.

Every protocol number here is read from the production Solidity, recomputed at build time by independent models, or quoted from the internal audit record, and the build fails if a stated number disagrees with the tested model. Where this paper and deployed bytecode disagree, the bytecode controls. The claim-by-claim record lives in `docs/whitepaper/FACT-CHECK.md`.

EDITION

v0.3, 10 August 2026. Supersedes v0.2. This edition describes the internally reviewed implementation rather than a target design: the contracts now match the surface this paper documents.

DOCUMENT STATUS

DOCUMENT VERSION

v0.3 · 10 August 2026

CONTRACTS DESCRIBED (COMMIT)

5ad1ebc50f2963c54593430036d384221e0bc10a

CODE CHANGED AFTER THAT CANDIDATE?

Yes: checked increments in Fundraiser.settleEpochs and the ADR 0022 fee-harvest redesign. The finalized campaign covers the later tree; the exact final commit has no separate re-review.

MAINNET DEPLOYMENT

Not deployed and not authorized for user funds

SECURITY REVIEW

Internal adversarial review only. Independent external audit: Not completed.

LEGAL REVIEW

Not performed

AUTHOR

Heesho

INTERNAL-REVIEW CANDIDATE (COMMIT)

54e3f2c3ce1de25aea4da2f21fab27804a3bfa84

INTENDED NETWORK

Robinhood Chain (chain ID 4663)

TESTNET DEPLOYMENT

None recorded in the repository

LICENSING AND PROVENANCE

Repository license and upstream provenance unresolved; distribution and deployment blocked pending counsel review

KNOWN OPEN FINDING

A-09, Medium: carry can reach later signal weight. Chapter 30.

One-paragraph disclaimer: this whitepaper explains open-source smart-contract software under development. It offers nothing for sale, promises no profit, guarantees no outcome, and omits any number it could not verify. Participation in any future deployment would carry real risks, described honestly in Part XI, and should never be based on this document alone.

HOW THIS PAPER IS ARRANGED

Contents

ORIENTATION

• How to read this paper	04
--------------------------	----

PART I · THE IDEA

01 Executive summary	05
02 Executive summary, continued	06
03 GumBall6900 in 90 seconds	07
04 The protocol in one figure	08
05 The core idea	09
06 A basket that forms over time	10
07 What "onchain" means here	11

PART II · TOKENS AND PARTICIPANTS

08 What USDG and GBX are	12
09 GBX lifetime supply	13
10 What sGBX is	14
11 Allocated versus idle sGBX	15
12 Who does what	16
13 Can, cannot, must trust	17

PART III · PUBLIC GBX MINING

14 What "mining" means here	18
15 The daily emission curve	19
16 The ideal curve versus the integer schedule	20
17 Sharing an epoch, and empty epochs	21
18 The mining market	22
19 Why mining profit is not guaranteed	23

PART IV · STAKING AND SIGNALING

20 Staking, step by step	24
21 Signals are absolute amounts, not percentages	25
22 Adding and removing, worked	26
23 What signals control - and killed Strategies	27
24 Under the hood: signal accounting	28

PART V · USDG REVENUE ALLOCATION

25 The revenue path	29
26 The revenue index and exact carry	30
27 Zero signal, killed Strategies, and distribution	31
28 The open finding: carry across weight changes	32

PART VI · STRATEGIES AND AUCTIONS

29 What a Strategy is - and which way the auction ...	33
30 The reverse Dutch auction, in plain English	34
31 Complete-lot pricing	35
32 Zero-price fills and the floor restart	36
33 Buyer protections and settlement	37

PART VII · STRATEGY REWARDS

34 Strategy rewards, and the word "Bribe"	38
35 Canonical and supplemental tokens, capped at ...	39
36 Reward streams, exactly	40

37 Selective claims, and the cost of eight	41
--	----

PART VIII · THE SHARED FUND

38 What the Fund is	42
39 Selective in-kind redemption	43
40 A worked redemption, and omitted assets	44

PART IX · LIQUIDITY

41 Genesis liquidity: 20 million GBX, one position	45
42 Permanent custody and one-time admission	46
43 Harvesting fees with fixed principal	47
44 Harvest timing, honestly	48

PART X · GOVERNANCE MINIMIZATION

45 What signals govern, what the manager controls	49
46 What the manager cannot do	50
47 The timelock, and a compromised key	51

PART XI · SECURITY AND TRUST

48 Security goals and layered protections	52
49 The supported-token model	53
50 The dependency map	54
51 What was actually tested	55
52 The finding register	56
53 What the protocol cannot guarantee	57

PART XII · USING THE PROTOCOL

54 Walkthroughs: mine, stake, signal	58
55 Walkthroughs: buy, claim, redeem, maintain	59
56 Safety checklists	60

PART XIII · QUESTIONS, ANSWERED

57 FAQ: the idea, mining, and signaling	61
58 FAQ: revenue, auctions, and rewards	62
59 FAQ: the Fund, governance, and status	63

PART XIV · CONCLUSION

60 The worked example, end to end	64
61 What GumBall6900 makes possible	65

APPENDICES

• Appendix A: Contract map	66
• Appendix B: Exact formulas	67
• Appendix C: Lifecycle summaries	68
• Appendix D: Access-control matrix	69
• Appendix E: Security invariants	70
• Appendix F: Evidence and toolchain record	71
• Appendix G: Residual risks and deployment che...	72
• Appendix H: Event guide	73
• Appendix I: Glossary	74
• Appendix J: References	75
• Legal and risk disclosure	76
• Document basis	77

ORIENTATION

How to read this paper

This is a long document because it is honest about details. You do not need all of it to get value from it.

If you have five minutes, read the two-page executive summary and the one-page figure that follows it. If you are deciding whether to participate someday, add Part II (what the tokens are), Part XI (security and trust), and the safety checklists in Part XII. If you are a developer or reviewer, the mechanism chapters in Parts III to X follow the contracts closely, and the appendices give the exact formulas, the access-control matrix, and the recorded test evidence.

Every major concept is introduced the same way: a plain sentence first, an everyday comparison, what the contracts enforce, and what they cannot guarantee. Technical detail sits in clearly marked *Under the hood* passages you can skip without losing the thread. Terms are defined before they are used; the glossary in the appendices collects them all.

One continuous worked example runs through the paper. Maya mines and signals, Elena signals, Noor fills auctions, and Leo redeems. All four are fictional, every asset in the example is illustrative, and every number in it is computed by the same arithmetic the contracts apply - never typed in by hand.

COLOR IS NOT DECORATION

Every figure uses one grammar. Blue is USDG capital arriving. Pink is the holder-directed chain: signal, the acquisition it causes, and optional rewards. Graphite is GBX supply and burns.

WHERE THE NUMBERS COME FROM

Charts are computed at build time from the same emission, auction, stream, and redemption rules the repository tests in TypeScript and Python. A stated number that drifts from the tested model fails the build.

■ USDG capital
Contributed and routed in

■ Signal, and what it buys
Weight, acquisition and independent reward

■ GBX supply and burns
Minted once, burned forever

The recurring callouts

In one sentence compresses a chapter. Why this matters connects a rule to its consequence. What the contract enforces and what this does not guarantee keep promises separate from hopes. Under the hood is optional depth. Important risk is never optional. Role notes - for miners, signalers, buyers, claimants, redeemers, and callers - flag what each reader should take away.

PART I · THE IDEA

1 Executive summary

GumBall6900 is an onchain fund whose holders continuously direct what it acquires next. This page and the next say everything essential.

GumBall6900 is a set of twelve immutable smart contracts that together run a shared investment pot, the **Fund**, on a public blockchain. Nobody manages the portfolio. Instead, the people who hold the protocol's token decide, continuously and in public, where the next dollar of inflow goes.

GBX is the protocol's token. It cannot be bought from the team, because there is no team allocation: 98% of all GBX that can ever exist is distributed through public *mining* - sending the stablecoin USDG to a contract called the **Fundraiser** and receiving a share of a fixed, publicly known daily emission. The remaining 2% seeds a permanently locked trading pool so GBX has a market at all. At most 1,000,000,000 GBX can ever be minted, and burning GBX never reopens that ceiling.

Signaling is how holders steer. Staking GBX one-for-one produces **sGBX**, a non-transferable receipt. Pointing an amount of sGBX at a **Strategy** - an eligible acquisition path, such as "accumulate a tokenized NVIDIA share" - raises that Strategy's share of the next USDG distribution. Signals are absolute amounts you add and remove independently, whenever you like, with no voting seasons and no lockups.

Acquisition happens by auction, not by oracle. Each Strategy periodically offers its accumulated USDG for sale and asks buyers to pay in the asset it wants; the required payment falls with time until someone accepts. Every completed payment - all of it - becomes backing in the shared Fund.

IN ONE SENTENCE

Mine GBX from a public schedule, stake it, point it at what the Fund should buy next, and redeem it for a slice of what the Fund already holds.

WHAT THIS IS NOT

Not a fixed index. Not a token pegged to net asset value. Not a DAO with proposals. Not audited, deployed, or authorized for user funds.

FOR EVERY READER

Chapters are self-contained. If a term is unfamiliar - blockchain, wallet, stablecoin, staking - Part II defines each one before relying on it.

Signals direct the future. The Fund records the past. Redemption connects the two.

PART I · THE IDEA

2 Executive summary, continued

What holders get back, what management can and cannot touch, and the risks that matter most.

Redemption is the exit. Burning GBX entitles the holder to a proportional, in-kind share of Fund assets they select themselves: burn 1% of supply, name the tokens you want, receive 1% of each. There is no redemption desk, no queue, and no pause switch - and equally no guarantee about what the received assets are worth.

Rewards for signalers exist but are never automatic. Each Strategy has a companion contract (technically named a **Bribe**; this paper says "Strategy reward") that can stream up to eight registered tokens to that Strategy's signalers - but only value someone independently deposits is ever paid. Auction proceeds never fund rewards.

Management is deliberately small. A timelocked owner can do exactly three ongoing things: add a Strategy, kill a Strategy, and register a reward token within a permanent cap of eight per Strategy. It cannot mint GBX, touch Fund assets, redirect payments, pause redemption, move the liquidity position, or upgrade any contract. The Fund and the liquidity contract have no owner at all.

The largest risks are stated, not hidden: immutable bugs would be permanent; assets the Fund holds can freeze, fail, or lose value; auctions can clear badly or not at all (a late fill can clear at zero); one internal Medium finding (A-09) about timing fairness remains open; and the system has had no independent audit. Part XI treats each of these in depth.

WHY THIS DESIGN EXISTS

Every removed power is a promise: no admin who can rug, no oracle to corrupt, no upgrade to sneak in. The cost is symmetrical - no admin who can fix, either.

IMPORTANT RISK

This software is unaudited and undeployed. Every property in this paper is an internal engineering claim until independent review says otherwise.

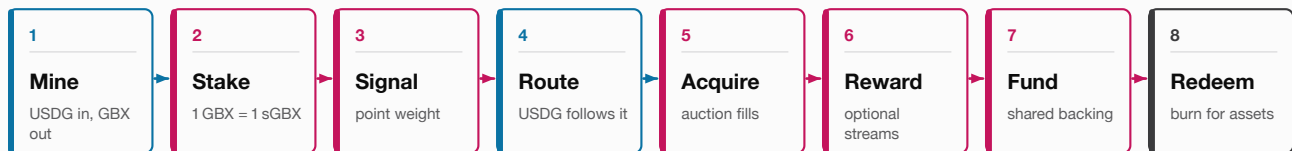
IF YOU ARE A... YOUR ONE-LINE TAKEAWAY

GBX miner	You are buying a share of a fixed daily emission; your cost depends on who shows up with you.
Signaler	Your sGBX steers future inflow only; it never sells what the Fund already holds.
Auction buyer	You choose the moment the price is right; three parameters protect your fill.
Reward claimant	Only independently funded streams pay; claim selectively if a token misbehaves.
Redeemer	You pick the assets; omitted claims are forfeited, and one snapshot prices everything.
Reviewer	Twelve direct contracts, three governed actions, one open finding, no external audit yet.

PART I · THE IDEA

3 GumBall6900 in 90 seconds

The whole lifecycle is eight permissionless moves. Everything after this page is detail and evidence.



Every move is a separate permissionless transaction. None is required to follow the last.

Steps 2, 3 and 6 are optional.

FIG. 01 Mine, stake, signal, route, acquire, reward, fund, redeem. Arrows are transaction flow, not obligations: each move is independent, optional after the first, and open to anyone.

A reader who has never used a blockchain can follow the loop in ordinary terms. People buy in by contributing digital dollars on a published schedule (*mine*). They convert their tokens into voting weight that cannot be sold separately (*stake*) and point that weight at the assets they want accumulated (*signal*). The machine forwards arriving dollars accordingly (*route*), sells them at open, falling-price auctions for the wanted assets (*acquire*), optionally lets third parties tip the people who steered (*reward*), stores everything bought in a shared vault (*fund*), and lets anyone cash out their share of that vault in the assets themselves (*redeem*).

WHY THIS MATTERS

No step requires permission, identity, or an operator being awake. The protocol is a standing set of rules, not a service that someone runs.

No proposal season. No signaling epoch. No withdrawal lock on unallocated sGBX. Every distribution re-reads the signal that exists at that moment.

PART I · THE IDEA

4 The protocol in one figure

Three kinds of value move through GumBall6900. This is all of them, and every place they can end up.

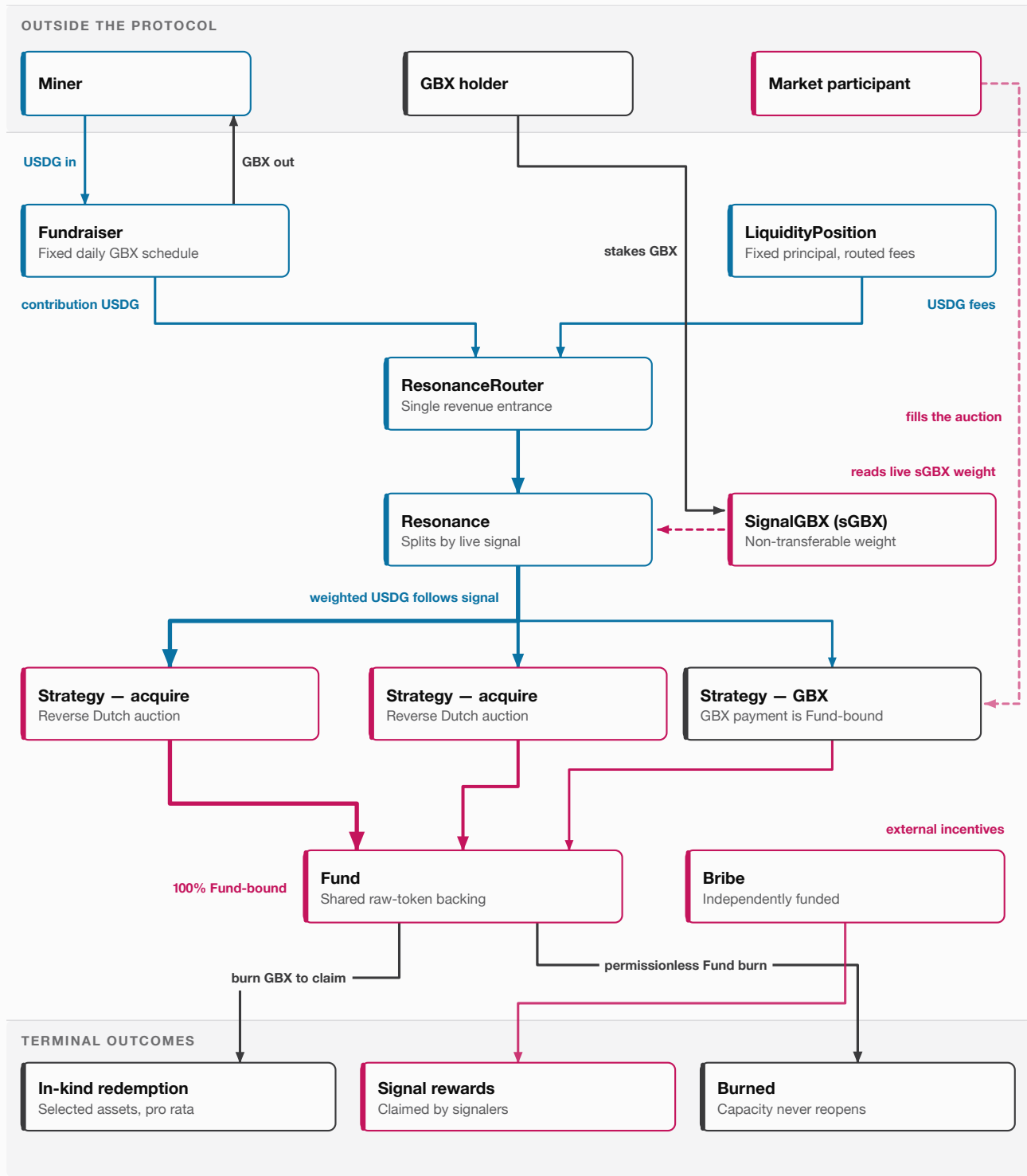


FIG. 02 Contribution revenue has exactly one entrance and follows live signal weight to a Strategy. Every completed Strategy payment is Fund-bound in full; rewards are only ever funded independently. Redemption and burning are the only exits, and burned mint capacity never reopens.

PART I · THE IDEA

5 The core idea

Funds usually make a basket easier to own. GumBall6900 makes the next purchase easier to influence.

A traditional index fund hands you a finished basket: a methodology decides membership, a committee decides weights, and you receive the result. Most onchain "index" products keep that shape and merely move custody onchain. GumBall6900 asks a narrower question instead:

Who should decide the fund's *next* purchase?

Its answer: whoever holds live signal weight, continuously, with no round to wait for. Holders never approve a target portfolio. They hold weight across eligible acquisition paths, and each new unit of revenue follows whatever distribution exists the moment it arrives. The consequence threads the whole design: the Fund is a record of past acquisitions, signals are a statement of present preference, and changing your mind redirects the future without forcing a single sale.

THE DISTINCTION TO KEEP

GBX is a claim on what the Fund already owns. **sGBX** directs what it may acquire next. They are deliberately different instruments.

MODEL	WHO PICKS ASSETS	REBALANCING	EXIT	WHERE GUMBALL6900 DIFFERS
Traditional index fund	A methodology	Automatic	Cash at NAV	No methodology, no NAV, in-kind exit
Fixed crypto index token	A curated list	Periodic	Trade or mint/burn	No fixed list; the basket forms over time
Managed treasury	A manager	Discretionary	Rarely direct	No manager can touch holdings at all
DAO treasury	Token votes on proposals	Per proposal	Vote-dependent	No proposals; continuous per-account weight
Synthetic index token	A price formula	By oracle	Against collateral	Real tokens held and redeemed, no oracle

The same three decisions - membership, weighting, timing - exist in every model above. What moves is where they are made and who can see them being made. Here, all three are relocated into public, continuous mechanisms: governance curates the menu, signals weight it, and buyers time it. Nothing in that sentence requires a vote, a season, or a quorum.

WHY THIS MATTERS

Relocating decisions is the whole product. Everything else in this paper is the supporting machinery that makes the relocation safe to run unattended.

PART I · THE IDEA

6 A basket that forms over time

Because every distribution follows the signal of its moment, the Fund holds the sum of every past decision - not a picture of the current one.

LIVE SGBX SIGNAL AT EACH DISTRIBUTION



RESULTING FUND COMPOSITION, CUMULATIVE

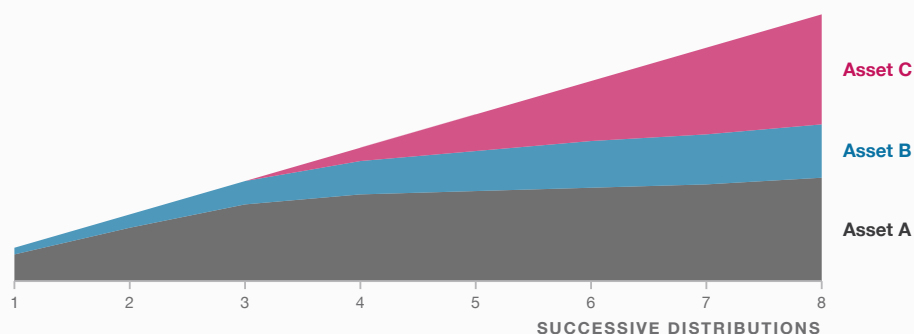


FIG. 03 The upper band is what holders want at each distribution. The lower area is what the Fund actually holds. They are related by accumulation and are never the same picture.

This is the most important behavioral fact in the design and the one most often misread. A holder who moves signal to a new Strategy has changed the destination of *future* capital and nothing else. Assets bought under the old signal stay in the Fund until someone redeems them out. There is no re-balancing engine, because building one would require exactly the discretionary selling power this design removes.

Composition therefore shifts fastest when inflow is large relative to holdings - early distributions shape the basket far more than later ones - and the one mechanism that shrinks an existing position is redemption, where a holder carries specific assets out with their burned GBX.

WHAT THIS DOES NOT GUARANTEE

The basket never automatically tracks current preferences, market weights, or any index. If everyone changes their mind, the Fund still holds what it holds.

SIMPLE EXAMPLE

A fund that signaled 100% NVDA for a year, then switches to 100% AAPL, still holds a year of NVDA. Only new dollars - and redemptions - change that.

PART I · THE IDEA

7 What "onchain" means here

A blockchain is a shared computer whose past cannot be quietly edited. Rules deployed to it run exactly as written - and only the rules that are actually on it.

For a reader new to this world: a **blockchain** is a public ledger maintained by many computers at once, so no single party can rewrite history or block a valid transaction. A **smart contract** is a program stored on that ledger; once deployed without upgrade hooks, its rules are fixed. A **wallet** is the keypair you act with, a **transaction** is a signed instruction, and **gas** is the execution fee the chain charges. GumBall6900 is intended for **Robinhood Chain**, an Ethereum-compatible network (chain ID 4663) whose support for the storage feature the Fund uses (EIP-1153) was verified read-only at a pinned block during internal review.

The figure divides the world honestly. Everything on the left is enforced by contract code this paper can cite. Everything on the right is real infrastructure the system needs but cannot control - including the issuers of every external token the Fund might hold.

WHY THIS MATTERS

"Onchain" is a statement about enforcement, not safety. A rule being immutable does not make it a good rule, and it does not protect assets whose own issuers keep offchain powers.

The paper's guarantees stop at this line	
ENFORCED ONCHAIN	OFFCHAIN OR EXTERNALLY DEPENDENT
• GBX supply, mint ceiling, burns • Contributions, emissions, claims • Staking, absolute signals, weights • USDG routing and allocation • Auction prices and settlement • Reward streams and claims • Fund balances and redemption • LP custody and fee harvesting	• Frontend, subgraph, RPC, wallet • Token metadata and asset discovery • Market prices and liquidity • Each token issuer's own controls • Legal rights behind wrapped assets • The chain itself and its operators • Independent audits (none yet) • Your own key management

FIG. 04 The left column is what this paper can promise. The right column is what it can only describe.

PART II · TOKENS AND PARTICIPANTS

8 What USDG and GBX are

One token comes in; one token measures your stake in what came in.

USDG is a stablecoin: a token issued by an external company that aims to hold the value of one US dollar and uses six decimal places (the smallest unit is \$0.000001). It is an **ERC-20** - the standard interface fungible tokens share, defining balances, transfers, and approvals. USDG is the only asset the Fundraiser accepts and the only asset Resonance allocates. Its issuer, like most stablecoin issuers, retains powers this protocol cannot see or veto: freezing addresses, upgrading the token, blocking transfers. That external trust is stated here once and revisited in Part XI.

GBX ("GUM BALL 6900") is the protocol's own 18-decimal ERC-20. It is transferable and carries two standard extras: *Permit* (signature-based approvals, EIP-2612) and *Votes* (balance checkpoints). Nothing in the protocol runs token voting; the checkpoint surface is inherited from the standard library and left available to others. GBX has exactly three protocol roles: it is what mining pays out, what staking converts into signal capacity, and what redemption burns.

GBX is *not* a claim to a fixed price. It is not pegged to the Fund's value and no oracle computes a "share price." What a GBX is worth is decided by markets and by what redemption can actually withdraw - nothing else.

IN ONE SENTENCE

USDG is the fuel; GBX is the share-like token that mining mints, staking locks, and redemption burns - with no promised price.

WHAT THE CONTRACT ENFORCES

Exact-transfer checks guard every USDG movement: if a token moves more or less than requested, the transaction reverts rather than mis-accounting.

WHAT THIS DOES NOT GUARANTEE

Neither token's market value. USDG can depeg or freeze; GBX can trade anywhere, including far from any notion of backing.

PART II · TOKENS AND PARTICIPANTS

9 GBX lifetime supply

One number is capped forever: how much GBX has ever been minted. Everything else follows from it.

LIFETIME MINT CEILING

1,000,000,000 GBX**980,000,000 GBX — public Fundraiser mining capacity, 98%****20,000,000 GBX — genesis liquidity only, 2%**

0 GBX to team, founders, investors, presale or advisors

The contract tracks minting and burning separately, and the distinction matters. **Lifetime minted** counts every GBX ever created, including tokens later destroyed; it can only rise, and it can never exceed one billion. **Lifetime burned** counts every GBX ever destroyed. **Current supply** is the difference, and **remaining mint capacity** is the ceiling minus lifetime minted - so burning GBX shrinks supply but never refills the mint. A billion minted and a billion burned would leave zero supply and zero capacity, permanently.

Minting authority is equally rigid: the deployer hands it to the Fundraiser exactly once, the handover locks, and from then on only Fundraiser claims can mint - within the 980 million reserved for them.

SIMPLE EXAMPLE

Think of a ticket roll printed with one billion serial numbers. Tearing tickets up does not print new ones; it only makes the surviving tickets a larger share of what remains.

QUANTITY	MEANING	CAN IT DECREASE?	ENFORCED WHERE
Lifetime minted	Every GBX ever created	Never	GBX.mint reverts past the ceiling
Lifetime burned	Every GBX ever destroyed	Never	GBX.burn accumulates it
Current supply	Minted minus burned	Yes, by burning	Standard ERC-20 accounting
Remaining capacity	1,000,000,000 minus lifetime minted	Only toward zero	remainingMintableSupply()

The split is exact by construction: 20,000,000 GBX of genesis liquidity plus 980,000,000 GBX of Fundraiser capacity equals the 1,000,000,000 ceiling, and the contracts assert that partition rather than assuming it.

FOR GBX HOLDERS

Every burn - buyback-style Strategy fills, harvested liquidity fees, redemptions - concentrates the remaining supply's claim on the Fund.

PART II · TOKENS AND PARTICIPANTS

10 What sGBX is

Staking converts GBX into signal capacity, one for one. The receipt moves influence out of the market.

Staking here means depositing GBX into the SignalGBX contract, which holds it and mints the same number of **sGBX** ("Signal GUM BALL 6900") to you. sGBX is **non-transferable**: any transfer that is not a mint or burn reverts. You cannot sell it, lend it, or delegate it away by accident; the only way out is *unstaking*, which burns sGBX and returns the same amount of GBX immediately.

Why forbid transfers? Because sGBX is influence over shared money. A transferable signal token would become a market in influence, rentable by parties with no exposure to the outcome. Binding weight to the staked position keeps the person steering the Fund and the person exposed to the Fund the same person.

There is deliberately no lock, no cooldown, and no unbonding period. The only thing that pins staked GBX in place is your own allocated signal - covered next - and even that only pins the allocated amount, never your whole balance.

IN ONE SENTENCE

sGBX is your steering weight: minted 1:1 by staking, unusable as money, and redeemable back into GBX the moment it is not actively pointing at something.

UNDER THE HOOD

Both stake and unstake verify exact GBX movement on both sides of the transfer, and unstake checks your allocated weight through Resonance before releasing.

WHAT THIS DOES NOT GUARANTEE

Staking earns nothing by itself. There is no staking yield; only allocated signal can ever attract independently funded rewards.

One GBX in, one sGBX out - and back again, immediately, for anything you have not allocated.

PART II · TOKENS AND PARTICIPANTS

11 Allocated versus idle sGBX

Holding sGBX does nothing by itself. Only the amounts you explicitly point at Strategies exist, as far as routing is concerned.

Maya's 1,500 sGBX



Idle sGBX earns nothing, directs nothing, dilutes nobody - and can be unstaked immediately. Allocated sGBX stays staked until its signals are removed.

FIG. 05 Maya's account from the worked example: 900 sGBX pointed at one Strategy, 400 at another, 200 idle. The idle 200 can be unstaked in the same block.

The protocol tracks, per account, an *allocated* total - the sum of your per-Strategy signals - and treats everything above it as *idle*. Idle sGBX earns no rewards, directs no USDG, and, just as important, does not dilute anyone: allocation math divides by total *allocated* weight, not by sGBX supply. An account that stakes a million GBX and points none of it is invisible to routing.

Idle weight is also instantly liquid. Unstaking checks only that the requested amount fits within your unallocated balance; it never requires touching your signals, waiting for an epoch, or paying anyone.

FOR SIGNALERS

Keep a working buffer idle if you like - it costs nothing, dilutes nothing, and leaves instantly. Allocate only what you mean to steer with.

WHY THIS MATTERS

Systems that count unallocated stake in the denominator quietly tax inattentive stakers. This one does not: absent weight simply does not exist to the router.

PART II · TOKENS AND PARTICIPANTS

12 Who does what

Most protocol actions are open to any address. The exceptions are one narrow timelocked owner and the external parties nobody can bind.

Contributor / miner Sends USDG, claims the epoch's GBX	GBX holder Holds, trades, burns, or redeems	Staker / signaler Stakes into sGBX, points weight	Auction buyer Fills a Strategy's USDG lot
Reward notifier Independently funds a registered token	Reward claimant Claims accrued streams	Redeemer Burns GBX for selected assets	Maintenance caller Settles, routes, harvests, burns, pays liabilities
Manager (timelock) Adds or kills Strategies, registers reward tokens	Token issuer Controls each external asset's own rules	Frontend / subgraph Presentation only; no protocol authority	You, reading this Verify before you interact

FIG. 06 Roles are hats, not registrations: one address can wear several. Contribution can even name a different beneficiary, so a service can mine on a user's behalf without ever holding their GBX.

Three roles deserve emphasis. The **maintenance caller** is anyone willing to pay gas to advance public state: settling Fundraiser epochs, routing revenue, distributing to Strategies, paying fixed Fund liabilities, harvesting liquidity fees, burning Fund-held GBX. None of these pay a bounty; the system assumes interested parties exist. The **manager** is a timelocked contract with three ongoing powers, described in Part X. And **token issuers** - of USDG and of every wrapped asset - hold powers entirely outside this system's reach.

IMPORTANT RISK

Unpaid maintenance is a real liveness assumption: if nobody volunteers gas, settlements, distributions, and harvests simply wait.

PART II · TOKENS AND PARTICIPANTS

13 Can, cannot, must trust

For each party: what the contracts let them do, what the contracts forbid, and what you are trusting anyway.

PARTY	CAN (ENFORCED)	CANNOT (ENFORCED)	YOU MUST STILL TRUST
Any address	Mine, stake, signal, buy, claim, redeem, and run every maintenance call	Touch another account's balances, signals, or rewards	That volunteers actually perform unpaid maintenance
Manager (timelock)	Add a Strategy, kill a Strategy, register reward tokens up to eight	Mint GBX, move Fund assets, redirect payments, pause, upgrade, migrate	Multisig signers not to schedule harmful-but-legal actions
Multisig signers	Propose and cancel timelocked actions	Execute without the public delay elapsing	Their key hygiene and their judgment
USDG issuer	Everything its own token contract allows	Nothing - this protocol cannot restrict it	Solvency, honesty, and not freezing protocol addresses
Wrapped-asset issuers	Freeze, upgrade, restrict, or halt their tokens	Nothing - same as above	Custody of the underlying and the legal wrapper terms
Frontends and indexers	Display anything they choose	Change onchain state or intercept funds	That what they show matches the chain - verify independently
The chain itself	Order, include, and execute transactions	Rewrite finalized history without consensus	Robinhood Chain's operators, uptime, and EIP-1153 support

Two vocabulary notes for later chapters. A **multisig** is a wallet that requires several signers to approve any action. A **timelock** is a contract that executes approved actions only after a public delay, giving everyone time to see - and react to - what is coming.

IN ONE SENTENCE

The contracts bind the operator tightly, the market loosely, and external issuers not at all.

PART III · PUBLIC GBX MINING

14 What "mining" means here

Mining is contributing USDG through the Fundraiser and receiving a pro-rata share of that day's fixed GBX emission. No hardware, no puzzles.

Each day is an **epoch**. The contract publishes, in advance and forever, how much GBX each epoch emits. Everyone who contributes USDG during an epoch splits that epoch's emission in proportion to what they contributed - a thousand USDG among forty thousand earns exactly one fortieth of the day. The smallest accepted contribution is 10,000 raw USDG units, which is one cent at USDG's six decimals.

Contributions are working capital from the first block: the USDG moves straight through the router into Resonance in the same transaction, where current signals decide which Strategies it feeds. There is no treasury wallet where it rests and no discretionary step between paying in and being routed.

A contribution names a **beneficiary**, which does not have to be the payer - a service can mine on your behalf while the claim stays yours. Claims are per-epoch, one per account, minted directly to the beneficiary, and never expire once the epoch settles.

IN ONE SENTENCE

You are not buying GBX at a price; you are buying a share of a fixed daily print run whose cost depends on who shows up beside you.

SIMPLE EXAMPLE

A bakery gives away a fixed batch of bread daily, split by how much flour each person brought. Bring flour on a quiet day and you take home more bread.

WHAT THE CONTRACT ENFORCES

The schedule, the pro-rata split, the routing, and the mint - all fixed. What it cannot enforce: that anyone shows up, or that GBX is worth anything.

Every contributed USDG is committed to the signal-directed path - never to a team wallet.

PART III · PUBLIC GBX MINING

15 The daily emission curve

The schedule's two constants are one decision: the decay is a four-year half-life, and the opening emission is derived so the schedule pays out its own allocation.

THE SCHEDULE FUNDS ITSELF

$$E_0 = A \times (1 - d), \quad \text{so} \quad \sum_{t \geq 0} E_0 \cdot d^t = A$$

$A = 980,000,000$ GBX mining allocation · $d = 0.999525354337060160$ per day (the 1,460-day half-life factor) · $E_0 = 465,152.749681042811702004$ GBX. In ideal real-number arithmetic the infinite series sums to the allocation exactly.

WHY A HALF-LIFE

A geometric decay needs no schedule table, no governance dial, and no end-date decision - one multiplier per day does all of it.

FOR GBX MINERS

The curve is common knowledge, so your edge is never information about supply - only your read on demand and participation.

Day one emits about 465,153 GBX. Four years in, the daily emission is half that; eight years in, a quarter. Each four-year period closes half the remaining distance to the allocation: roughly 50% emitted by year four, 75% by year eight, 93.75% by year sixteen. Front-loading is deliberate - early participation carries the most weight - but the tail is long enough that mining never has a cliff.

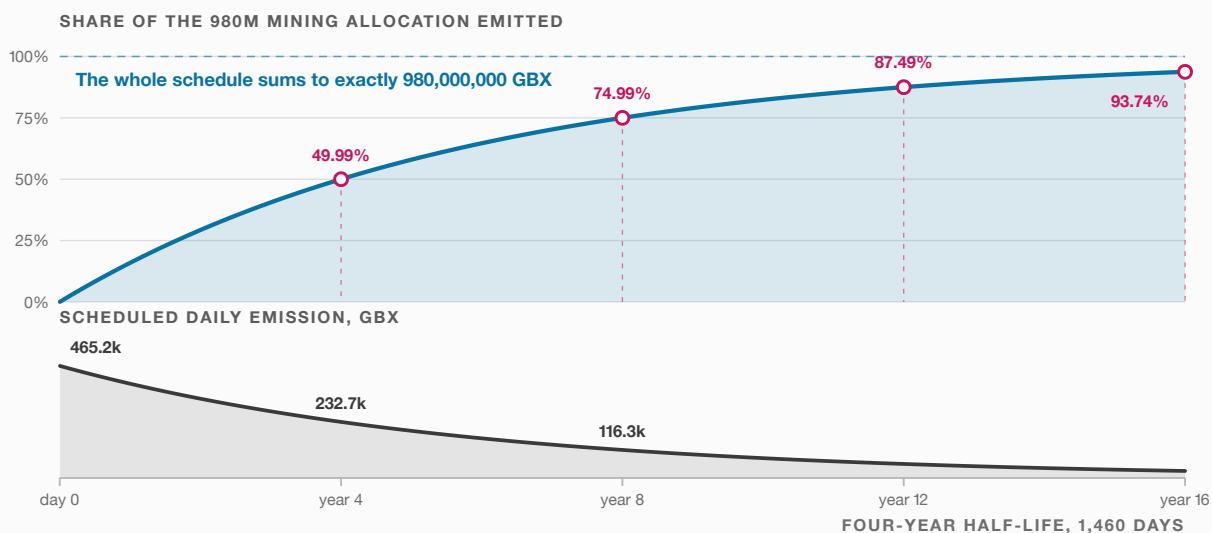


FIG. 07 The ideal curve. Cumulative emission approaches - never exceeds - the 980 million allocation, halving its remaining distance every four years. The next chapter shows where the real integer schedule departs from this ideal.

PART III · PUBLIC GBX MINING

16 The ideal curve versus the integer schedule

Contracts do not compute with real numbers. The onchain schedule floors after every daily step - and that changes the ending.

The contract stores each day's emission as a whole number of wei (10^{-18} GBX) and computes the next day as $\text{floor}(\text{current} \times d / 10^{18})$, one sequential floor per day. Each floor discards a fraction of a wei, so the integer schedule runs slightly below the ideal curve - and unlike the ideal curve, it terminates. Replaying all of it, in two independent implementations cross-checked against the repository's tested fixture, gives exact endpoints:

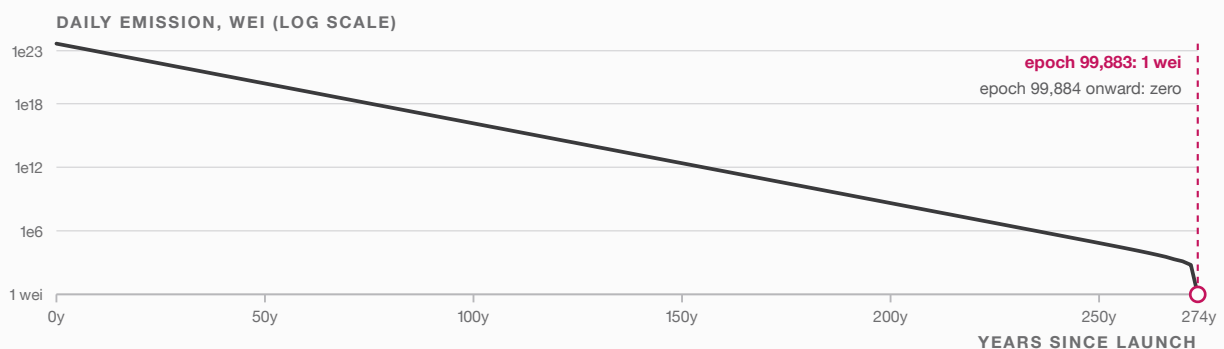
FACT	EXACT VALUE
First emission (epoch 0)	465,152.749681042811702004 GBX
Decay factor per day	0.999525354337060160
Nominal half-life	1,460 days (four years)
Nonzero epochs	99,884 (the last is epoch index 99,883, about 273 years in)
Final nonzero emission	exactly 1 wei
Total if every epoch is claimed	979,999,999.9999991815005172 GBX
Unminted rounding remainder	818,184,994,828 wei ≈ 0.0000008 GBX

WHY BE THIS PRECISE

Earlier drafts ascribed the ideal curve's endlessness to the contract itself. False: the onchain schedule ends. This paper states both curves and never substitutes one for the other.

UNDER THE HOOD

The remainder is a deterministic consequence of flooring, not a reserve: no address, including governance, can ever mint it.



Total nonzero epochs: 99,884 · emitted if every epoch is claimed: 979,999,999.999999 GBX · unminted remainder: 818,184,994,828 wei

FIG. 08 The real schedule on a log axis: a straight geometric slide from $\sim 4.65 \times 10^{23}$ wei per day to a final single-wei epoch, then exactly zero forever.

PART III · PUBLIC GBX MINING

17 Sharing an epoch, and empty epochs

One formula splits each day. One rule handles a day nobody showed up for: its emission is gone.

A CONTRIBUTOR'S CLAIM

$$\text{reward} = \text{floor}(\text{contribution} \times \text{epochEmission} / \text{totalEpochContributions})$$

Worked: Maya contributes 1,000 USDG on day 121, which is scheduled to emit 439,393.270416 GBX. The day closes at 40,000 USDG total, so her claim is $\text{floor}(1,000 / 40,000 \text{ of the emission}) = 10,984.831760 \text{ GBX}$.

WHY FORFEIT, NOT ROLL OVER

Rollovers would make quiet stretches mint windfalls for whoever times the recovery - and would turn the clean schedule into a lottery about other people's absence.

FOR GBX MINERS

Claims never expire once settled, but nothing is minted until someone claims. Check `pendingReward(epoch, account)` before paying gas.

Settlement is sequential and permissionless: anyone may call `settleEpochs(maximum)` to advance ended epochs in strict order, applying exactly one decay step per epoch, in caller-sized batches so a long quiet stretch never becomes an unseizable backlog. An epoch with zero contributions settles like any other - except its emission is recorded as zero for contributors. It does not roll into tomorrow, it is not held for later, and no future contributor can ever claim it. The schedule advances on wall-clock time, indifferent to attendance.

Note the distinction between *lifetime minted* and what this chapter implies: empty epochs and claim-level flooring mean actual lifetime minting can end below the schedule's own total. The ceiling is a maximum, not a destiny.

QUIET DAY

10,000 USDG competes

Total contributed

10,000 USDG

1,000 USDG earns

43,939.32 GBX

of 439,393.27 GBX scheduled

CROWDED DAY

40,000 USDG competes

Total contributed

40,000 USDG

1,000 USDG earns

10,984.83 GBX

of 439,393.27 GBX scheduled

EMPTY DAY

Nobody contributes

Total contributed

0 USDG

1,000 USDG earns

nothing to claim

of 439,393.27 GBX scheduled

The schedule advances on wall-clock time. The empty day's emission is forfeited forever.

FIG. 09 The same 1,000 USDG contribution against the same scheduled emission: quiet days pay multiples of crowded days, and an empty day pays no one at all.

PART III · PUBLIC GBX MINING

18 The mining market

The Fundraiser never sets a price. It hands a fixed daily quantity to whoever showed up - which makes the crowd itself the price.

Because the day's emission E is fixed, the average cost every miner pays is simply C / E - total USDG contributed over total GBX emitted - no matter who contributed what. If GBX trades at price P with real, exit-sized liquidity, the day's emission is worth about $P \times E$, and mining looks attractive while contributions sit below that. Entry then closes the gap, which gives the rough break-even benchmark $C \approx P \times E$. The adjustment runs both ways: a thin day is precisely the day worth entering.

READ THE CHART THIS WAY

The rising line is what miners pay on average. The dashed line is what GBX quotes at. The wedge between them attracts the next miner - and closes as they arrive.

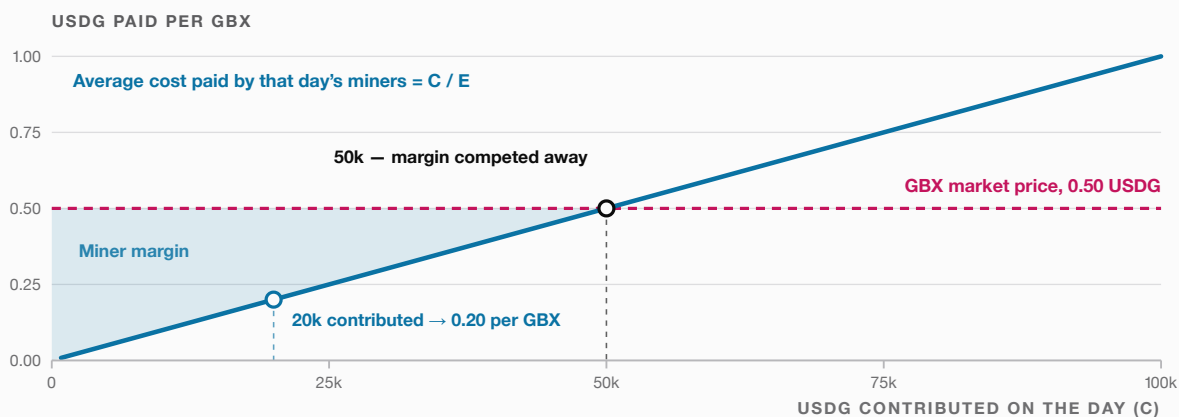


FIG. 10 An illustrative day: 100,000 GBX emitted, GBX quoted at 0.50 USDG. At 20,000 USDG contributed the average cost is 0.20; competition pushes toward 50,000, where the simple spot margin is gone. Profitable entry below the line exists only before gas, slippage, and risk.

WHAT THE CONTRACTS GUARANTEE

- Every contributed USDG enters the signal-directed path, never a discretionary wallet.
- Miners compete for a fixed public schedule, not a team-selected sale price.
- Every completed Strategy payment is owed entirely to the Fund.

WHAT NO CONTRACT CAN GUARANTEE

- That a quoted GBX price survives your exit size, or that liquidity exists at all.
- That mining is profitable on any given day - or ever.
- That auctions clear, or that Fund assets hold value.

PART III · PUBLIC GBX MINING

19 Why mining profit is not guaranteed

The break-even benchmark is a compass, not a promise.
Between it and your outcome sit at least eight real costs.

COST OR RISK	HOW IT BITES A MINER
Late competition	Contributions arriving after yours dilute the whole day, including you - the day's cost is known only at midnight.
Realizable price	The quoted GBX price is not your exit price. Thin liquidity means slippage; heavy selling moves the market itself.
Gas	Contributing, claiming, staking, and exiting each cost transaction fees regardless of outcome.
Timing	Your USDG routes to Strategies immediately, but your GBX arrives only after the epoch ends and settles.
Smart-contract risk	Unaudited, immutable code. An exploit or flaw can be permanent.
Stablecoin risk	USDG can depeg, freeze, or restrict transfers under its issuer's own rules.
Market risk	GBX's value depends on demand for the whole mechanism, not on the schedule.
Regulatory risk	Token distribution programs face uncertain and jurisdiction-dependent treatment.

IMPORTANT RISK

The equilibrium story describes incentives, not results. Days can stay unprofitable, or profitable, far longer than any model implies - and the model is not the market.

FOR GBX MINERS

A disciplined check: today's emission, total contributed so far, quoted price, and the depth behind that quote. Then decide - knowing the day is not over.

The contract fixes what happens [after](#) you contribute. Whether contributing was wise is entirely yours.

PART IV · STAKING AND SIGNALING

20 Staking, step by step

The staking contract is deliberately boring: hold deposits exactly, mint receipts exactly, release instantly when nothing is allocated.

`stake(amount)` pulls GBX with exact-transfer verification on both sides - the contract checks that your balance fell by precisely the requested amount and its own rose by the same - then mints equal sGBX. First-time stakers are self-delegated automatically so the standard checkpointing works without a second transaction. `unstake(amount)` runs the same choreography in reverse, guarded by a single condition: the amount must fit within your *unallocated* balance, your sGBX minus the sum of your live signals.

In the worked example, Maya stakes 1,500 GBX, allocates 900 and 400 to two Strategies, and immediately unstakes her idle 200 - in the same block, with no notice period. Her allocated 1,300 stays staked exactly until she removes those signals, and not a moment longer.

There is no reward for staking itself, no inflation directed at stakers, and no penalty for leaving. Staking is a capability, not a yield product.

IN ONE SENTENCE

Staking converts money into steering weight at par, and back again on demand for whatever is not actively steering.

UNDER THE HOOD

Unstake asks Resonance for `accountSignalWeight` and refuses only amounts that would dip into allocated weight. No other contract can freeze or seize a staked balance.

WHAT THIS DOES NOT GUARANTEE

sGBX is exposed to GBX's market risk the whole time. Non-transferability protects governance, not price.

PART IV · STAKING AND SIGNALING

21 Signals are absolute amounts, not percentages

The contract stores "900 sGBX on NVDA" - never "60% of my account". The difference decides how everything else behaves.

ADD ABSOLUTE AMOUNTS; LEAVE ANY REMAINDER IDLE

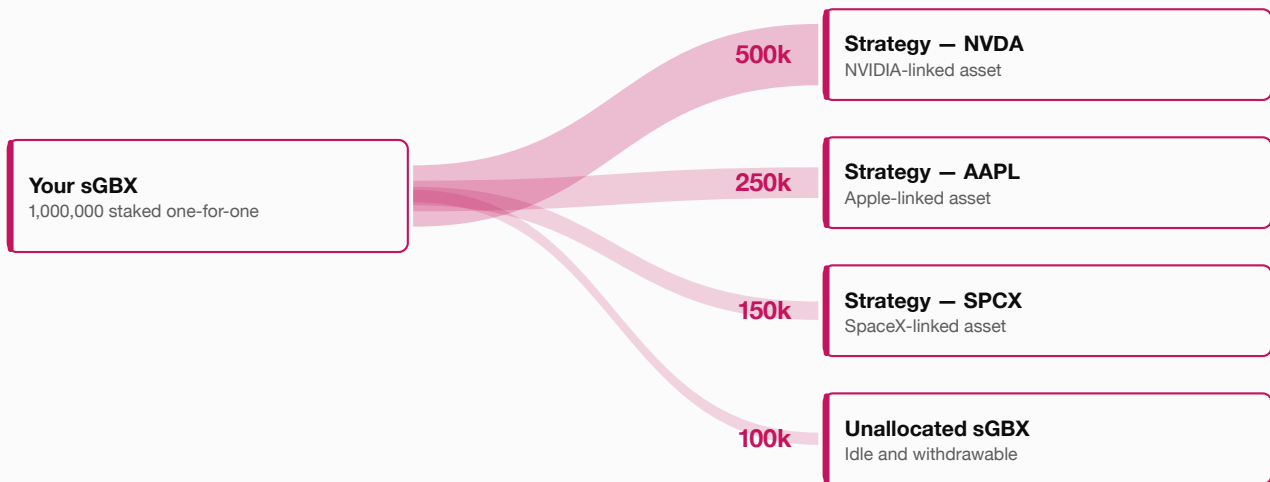


FIG. 11 A live allocation across several accounts. Each bar is an absolute per-Strategy amount; a frontend may display percentages for convenience, but the chain stores amounts.

Four functions manage signals, all taking absolute sGBX amounts as *deltas*: `addSignal(strategy, amount)` and `removeSignal(strategy, amount)` adjust one Strategy; `addSignalMany` and `removeSignalMany` apply caller-chosen batches. Adding checks only your unallocated balance; removing checks only that Strategy's existing signal. Nothing ever rescales your other positions, and there is no operation - none - that resets a whole account.

An earlier design used relative whole-account signals, where changing anything meant restating everything. It is gone, and this paper documents only the shipped model: additions and removals are independent, per-Strategy, and composable in any order.

WHY THIS MATTERS

Absolute deltas make costs local: touching one Strategy never loops over your others, so gas scales with what you changed, not what you hold.

FOR SIGNALERS

To move 150 from AAPL to TSLA: `removeSignal(AAPL, 150)` then `addSignal(TSLA, 150)` - or batch both in one call. NVDA never notices.

PART IV · STAKING AND SIGNALING

22 Adding and removing, worked

Maya trims one position by 150. Watch what changes - and the longer list of what does not.

BEFORE



The amount is a delta, not a target. The NVDA signal is untouched; the freed 150 sGBX becomes immediately withdrawable.

FIG. 12 `removeSignal(AAPL, 150)` lowers one allocation from 400 to 250 and frees 150 into the idle balance. The NVDA position, the account list, and every other account are untouched.

Every signal change also settles bookkeeping at the moment of the change: the global revenue index is brought current and the affected Strategy is checkpointed *before* weights move, so revenue that arrived under the old weights is allocated under the old weights. The Strategy's reward contract then mirrors the new balance. This ordering is why signal timing can never rewrite the past - a new signal participates only in value that arrives after it, subject to one carefully disclosed exception (the open A-09 finding, chapter 28).

Batches are caller-sized: the contracts never force you to iterate anything larger than the list you submitted. A signal reduced to zero also drops out of your account's tracked Strategy list - detail in chapter 24.

WHAT THE CONTRACT ENFORCES

Additions bound by unallocated balance; removals bound by the target's existing signal; checkpoint-before-change on every mutation; exact Bribe mirroring.

SIGNAL TIMING

Ordering within a block still matters: a distribution and a signal change in the same block settle in transaction order, like everything else onchain.

PART IV · STAKING AND SIGNALING

23 What signals control - and killed Strategies

A signal is one lever: the share of future USDG a Strategy receives. Everything else it might seem to control, it does not.

SIGNALS CONTROL

- Each live Strategy's share of every future USDG distribution.
- Your own share of that Strategy's independently funded rewards.
- Nothing else.

SIGNALS DO NOT CONTROL

- Which Strategies exist (governance) or when auctions fill (buyers).
- Anything the Fund already holds - no sale, no rebalance, ever.
- Reward funding (independent notifiers) or protocol parameters (none exist).

IDLE SGBX AND DILUTION

Only allocated weight is in any denominator. Idle sGBX neither earns nor dilutes - a fact worth internalizing before reading reward math.

FOR SIGNALERS

After a kill: your unrelated signals and idle balance were never at risk; remove the dead signal at leisure and re-point it live.

When governance **kills** a Strategy, the kill is forward-looking: the Strategy stops receiving future USDG, and revenue already indexed to it converts into a fixed liability owed to the Fund. Existing signals are never confiscated - they simply stop mattering, and their owners remove them whenever convenient. Adding to a killed Strategy reverts; removing from one always works, because removal makes no token transfer at all (the design consequence of resolved finding A-04). Even a Fund that temporarily rejects transfers cannot trap a signaler.

Exit liveness is absolute by construction: removing signal transfers no token, so no token can block it.

PART IV · STAKING AND SIGNALING

24 Under the hood: signal accounting

Skippable detail for reviewers: the exact state the contracts keep, and the identities the test suite holds invariant.

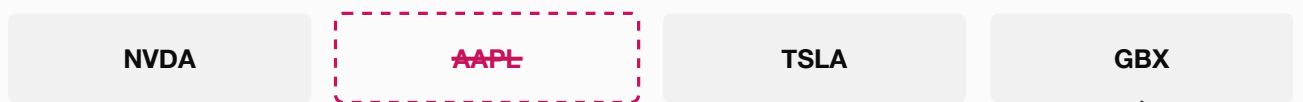
STATE	MEANING	HELD INVARIANT
accountSignals[a][s]	Account a's absolute signal on Strategy s	Sums over s to accountSignalWeight[a]
accountSignalWeight[a]	Account a's total allocated weight	Never exceeds a's sGBX balance
strategySignalWeight[s]	All signal pointed at Strategy s	Sums over accounts to the same total
totalSignalWeight	All allocated weight everywhere	Equals both sums; the routing denominator
accountStrategies[a]	The Strategies a currently signals	Swap-and-pop keeps it exact and duplicate-free
Bribe(s) virtual balances	Mirror of accountSignals for rewards	$\text{balanceOf}(a) = \text{accountSignals}[a][s];$ $\text{totalSupply} = \text{strategySignalWeight}[s]$

BRIBE VIRTUAL BALANCES

Reward contracts never see tokens moving; Resonance pushes weight changes into them so reward accounting always mirrors signal accounting exactly.

These identities were exercised by 13.5 million randomized state-machine calls with zero handler reverts during internal review (Part XI states the campaign precisely). They are engineering evidence, not an external audit.

ACCOUNT STRATEGY LIST BEFORE



AFTER: LAST ENTRY SWAPPED IN, LIST POPPED



Order is not meaningful; membership is. Removal costs the same wherever the entry sits.

FIG. 13 Removing a zeroed entry swaps the last list element into its slot and pops the tail - constant-time maintenance with no ordering promises.

PART V · USDG REVENUE ALLOCATION

25 The revenue path

All USDG revenue - contributions and harvested liquidity fees alike - crosses one router into one allocator, which holds no opinions.

SPLIT — BY LIVE SIGNAL WEIGHT
USDG changes hands

SETTLEMENT — 100% FUND-BOUND
and becomes the acquired asset

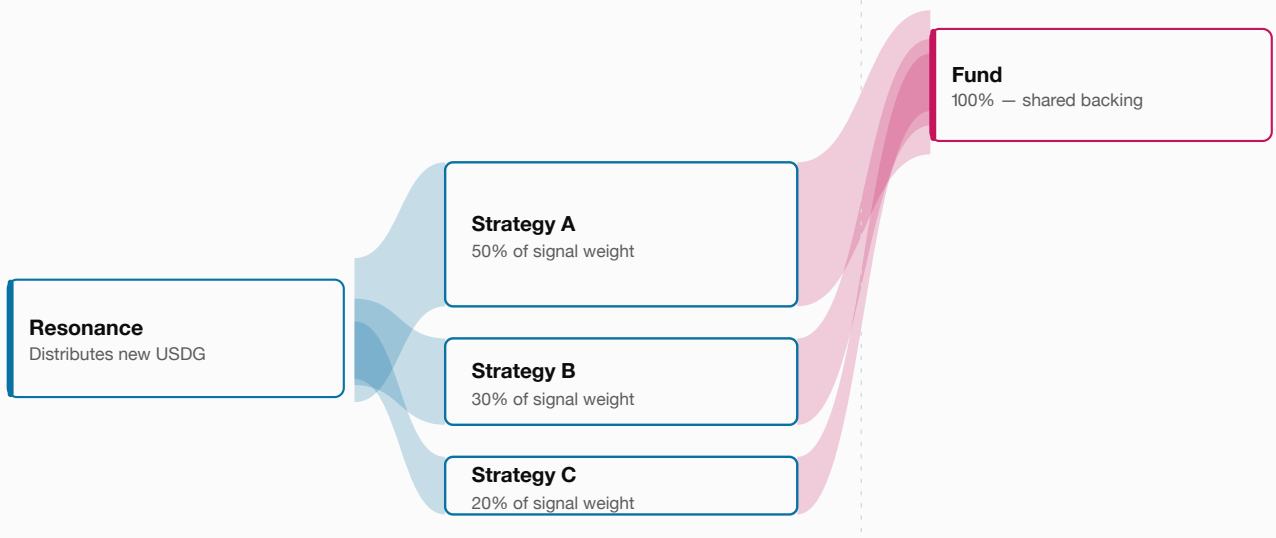


FIG. 14 Ribbon thickness is proportional to live signal share. Resonance reads the distribution that exists at the moment revenue arrives and applies it - nothing more.

ResonanceRouter is a fixed pipe: it accepts USDG from anywhere - the Fundraiser forwards every contribution to it, the liquidity position routes harvested fees to it, and anyone may simply transfer USDG at it - and its single permissionless function pushes the whole balance into **Resonance**. Resonance is the only contract allowed to account that revenue, and the router is the only address allowed to notify it. There is no second entrance, no discretionary wallet, and no pause between arrival and allocation.

Why a separate router at all? Because it makes "all revenue takes one path" a checkable property rather than a habit: whatever new revenue source ever appears, it can only join by transferring to the same entrance every other source uses.

IN ONE SENTENCE

Revenue in, weights read, USDG owed to Strategies - with the router existing so that sentence has no exceptions.

WHAT THE CONTRACT ENFORCES

Exact transfer deltas at every hop, a single authorized notifier, and fail-closed accounting: a route that cannot complete reverts entirely.

PART V · USDG REVENUE ALLOCATION

26 The revenue index and exact carry

Allocation is proportional to live weight - implemented as a global index with explicit remainders, so integer division never quietly deletes value.

CONCEPTUALLY

$$\text{strategyShare} = \text{revenue} \times \text{strategyWeight} / \text{totalSignalWeight}$$

Worked: 12,345.678901 USDG arrives against weights of 45,000 / 30,000 / 15,000 sGBX. The three Strategies accrue 6,172.83, 4,115.22, and 2,057.61 USDG; the sub-cent tail (0.018901 USDG) stays as exact carry for the next event.

UNDER THE HOOD

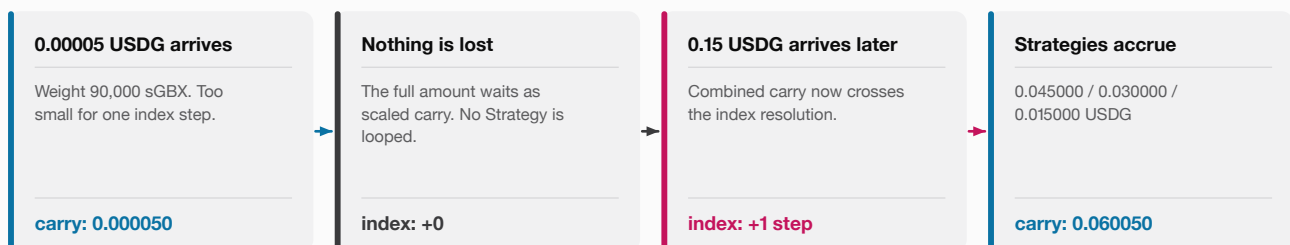
One identity is tested continuously:
 $\text{accountedBalance} \times 10^{18} = \text{pendingScaled} + \text{indexedScaled} + \sum \text{strategyRemainders} + (\text{claimable} + \text{fundLiability}) \times 10^{18}.$

WHY THIS MATTERS

Dust sounds trivial until millions of small contributions make it systematic. Exact carry means the protocol's books always balance to the wei.

Technically, Resonance maintains a **global revenue index**: cumulative USDG per unit of signal weight, at 10^{18} fixed-point precision. New revenue raises the index; each Strategy remembers the index at its last checkpoint and accrues $\text{weight} \times \text{index-growth}$ when touched. Division leaves remainders at two levels - revenue too small to move the global index, and per-Strategy sub-unit accruals - and both are stored, not discarded: a global scaled carry and a per-Strategy scaled remainder. Anyone can call `indexPendingRevenue()` to convert accumulated carry the moment it crosses one index step.

This is the resolved form of internal finding A-02: an earlier design floored per notification and could strand dust forever. The shipped design conserves every unit, with stateful invariant tests reconciling all carry classes against the contract's actual balance.



Numbers computed by the worked model at 90,000 sGBX of total weight. Anyone may call `indexPendingRevenue()` to convert waiting carry.

FIG. 15 A 0.00005 USDG arrival cannot move the index at 90,000 sGBX of weight, so it waits - exactly - until later revenue tips the combined carry over one step.

PART V · USDG REVENUE ALLOCATION

27 Zero signal, killed Strategies, and distribution

Revenue with no live weight to follow, and revenue indexed to a Strategy that has since been killed, both take the same fixed road: to the Fund.

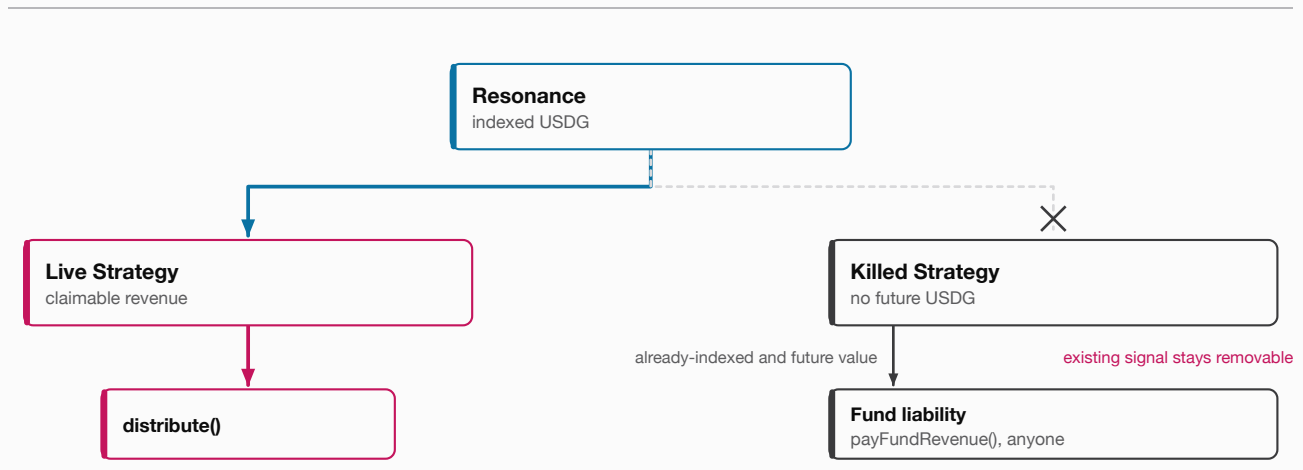


FIG. 16 Live Strategies accumulate claimable revenue that anyone can distribute to them. Killed-Strategy and zero-weight revenue accrues to a fixed Fund liability that anyone can pay.

If revenue arrives while *no* signal is allocated anywhere, there is nothing proportional to do - so whole units accrue to `fundRevenueLiability`, an irrevocable entitlement of the Fund that any caller can deliver via `payFundRevenue()`. Killing a Strategy converts its already-indexed, undistributed revenue the same way, and any later index growth attributable to its remaining signal weight follows suit. The destination is fixed in code; no key can redirect it.

Moving USDG to live Strategies is equally permissionless: `distribute(strategy)`, `distributeAll()`, or a bounded `distributeRange(start, end)` for gas-conscious callers. Direct USDG donations to Resonance are also handled honestly: they sit visibly as *unaccounted revenue* until anyone calls `syncRevenue()`, which folds them into the same carry accounting as routed revenue.

WHAT THE CONTRACT ENFORCES

Fixed destinations, permissionless delivery, and fail-visible accounting - a balance below the books' total reverts rather than papering over a hole.

FOR SIGNALERS

A killed Strategy's future is the Fund, not you. Your recourse is simply to re-point weight at Strategies you still believe in.

PART V · USDG REVENUE ALLOCATION

28 The open finding: carry across weight changes

Internal review left one Medium finding open, and it lives here.
This page is its complete, undiluted disclosure.

WHAT CONSERVATION PROVES

- 99 base units arrive; two incumbents hold all weight
- The 99 units wait as exact carry
- No unit is ever lost or double-counted

Solvency holds exactly.

WHAT A-09 PERMITS (OPEN FINDING)

- New signal doubles the weight after the value arrived
- 101 more units push the carry over the threshold
- The late weight shares value that predates it

Attribution across time is not guaranteed.

Bound: $\text{under totalSignalWeight} / 1e18 \text{ base units per bucket} - \text{negligible at 18 decimals, up to 1,000 whole tokens at 6.}$

FIG. 17 Conservation and attribution are different properties. The books always balance; who the balance belongs to across a weight change is the open question.

Carried value - both revenue carry in Resonance and reward carry in the Bribes - waits below index resolution until enough accumulates, then divides by the signal weight that exists *at that moment*. If weight changed between arrival and indexing, later entrants share value that predates them. Two deterministic proof-of-concept tests in the repository demonstrate exactly this: a Strategy signaled after 99 base units arrived can capture 100 of the eventual 200, and a signaler entering mid-carry can claim more than their pro-rata share of post-entry emission.

The bound is strict and small in most cases: less than $\text{totalSignalWeight} / 10^{18}$ base units per bucket - immaterial for 18-decimal tokens, but up to about 1,000 whole tokens for a 6-decimal token (USDG included) at the one-billion-sGBX extreme. Because signaling has no cooldown, the timing is permissionless and could be flash-assisted. No custody, solvency, or exit property is affected; this is an allocation-fairness defect, not a loss of funds.

It remains open because every exact fix - historical buckets, a dust destination on weight change, or explicit acceptance with decimal limits - changes storage, exit bounds, or economic policy, and that is an owner and independent-reviewer decision, not an engineering patch. Until resolved, this paper treats A-09 as a standing caveat on every proportional-allocation claim it makes.

IMPORTANT RISK

Open finding A-09 (Medium): pending carry can be temporally reallocated across signal changes. Worst case grows with signal supply and shrinking token decimals.

WHY DISCLOSE THIS HARD

A paper that only lists resolved findings is marketing. The one still open is precisely the one a prospective user deserves to see first.

PART VI · STRATEGIES AND AUCTIONS

29 What a Strategy is - and which way the auction points

A Strategy is a small immutable contract that accumulates USDG and sells it. Getting the direction right unlocks every other chapter in this part.

Each Strategy is created by governance with one immutable configuration: the **payment token** it wants (a tokenized stock wrapper, GBX itself, or any standard ERC-20), an opening price, an epoch length, a price multiplier, and a floor for future openings. From then on it does one thing: receive signal-directed USDG from Resonance and run a permanent sequence of auctions.

State the direction plainly, because intuition inverts it: **the Strategy is the seller**. It offers its entire current USDG balance as one lot. An external buyer - Noor, in our example - pays the required amount of the payment token and takes the USDG. The protocol ends up holding the payment token; the buyer ends up holding stablecoins. The Strategy never goes shopping on an exchange, never reads a price feed, and never decides a price is fair - the buyer's willingness is the price discovery.

A Strategy whose payment token is GBX is the buyback-style configuration: buyers pay GBX for the USDG lot, and that GBX becomes Fund property destined for burning (chapter 33).

IN ONE SENTENCE

A Strategy is a standing offer: "my USDG for your asset, on a price path that only moves down until someone says yes."

SIMPLE EXAMPLE

A vending machine in reverse: it holds cash, displays the number of tokens it wants for that cash, and lowers the ask every hour until a passerby accepts.

BOUNDED BY CONSTRUCTION

Epoch length is fixed between 1 hour and 365 days; the multiplier between 1.1x and 3.0x; configured prices between 10^6 and $2^{192}-1$ raw units.

The buyer brings the asset. The Strategy hands over the stablecoins. Both legs settle in one transaction.

PART VI · STRATEGIES AND AUCTIONS

30 The reverse Dutch auction, in plain English

A Dutch auction starts high and falls until someone accepts. Here the falling number is what the buyer must pay for the Strategy's whole USDG lot.

THE REQUIRED PAYMENT DURING AN EPOCH

$$\text{payment}(t) = \text{initial} - \text{floor}(\text{initial} \times \text{elapsed} / \text{duration})$$

Worked: the NVDA-linked Strategy opens an epoch asking 60 wrapper units for its lot. Seventeen hours into a 24-hour epoch, the ask has fallen to 17.5 units - the moment Noor decides that is cheap enough for 6,172.83 USDG, and fills.

After a fill at payment p , the next epoch opens at $p \times \text{multiplier}$ (2.0x in the example: the next open is 35 units), floored at the configured minimum. That ratchet is the whole pricing mechanism: fills that come early say the ask was too generous and push the next opening up; fills that come late, or not at all, walk it down. Across epochs the sequence hunts the market's actual valuation and re-tests it forever - with no oracle anywhere.

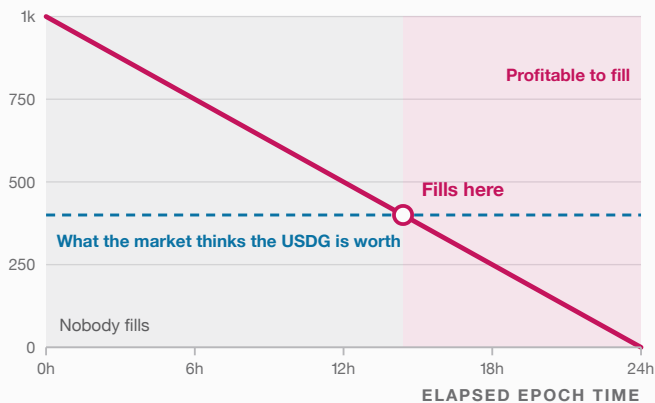
WHY NO ORACLE

An oracle is a trusted reporter of prices, and trusted reporters get corrupted, censored, or simply wrong at the worst time. A clock cannot be bribed.

WHAT THIS DOES NOT GUARANTEE

Oraclelessness removes a dependency, not market risk: thin participation or bad parameters still produce late, unfavorable, or missing fills.

ONE EPOCH — TARGET ASSET THE FILLER MUST PAY



FOUR EPOCHS IN SEQUENCE

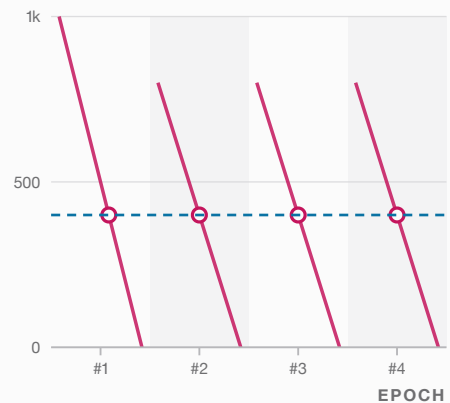
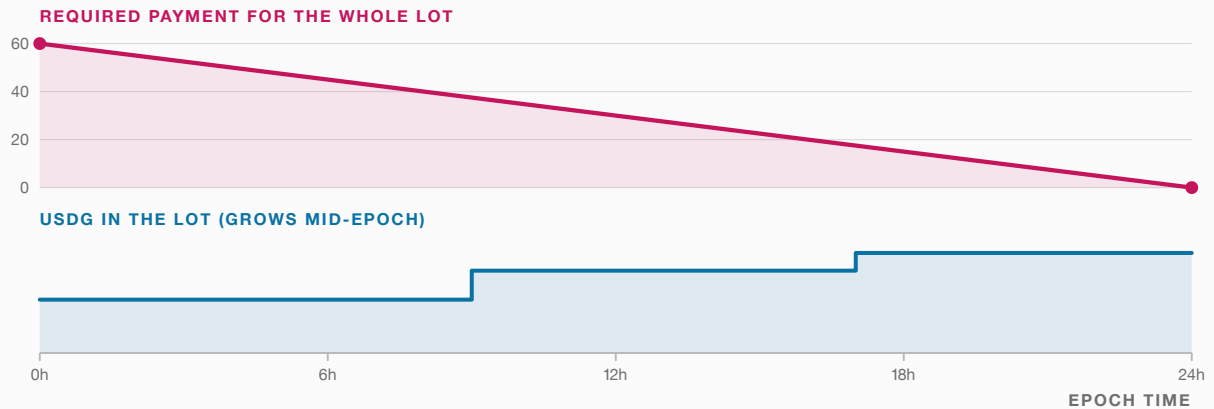


FIG. 18 Left: within an epoch the required payment decays linearly to zero; a rational buyer fills when it crosses their own valuation of the lot. Right: across epochs, each fill re-opens higher by the bounded multiplier, so the sequence tracks a moving market price from below.

PART VI · STRATEGIES AND AUCTIONS

31 Complete-lot pricing

`currentPrice()` quotes one number for the entire USDG balance - however large that balance has become since the epoch opened.



CONSEQUENCE

The clock prices the lot, not the token. More USDG entering mid-epoch makes the same payment buy more, so the effective rate is path-dependent.

FIG. 19 The payment follows only the clock. USDG entering mid-epoch enlarges the lot without repricing it, so the effective per-USDG rate depends on the deposit path - not just the fill time.

This is a genuine subtlety, stated without varnish: the auction quote is *not* a per-USDG price. A buyer watching an epoch is watching two numbers move independently - the required payment falling with time, and the lot growing with each distribution that lands mid-epoch. More USDG arriving makes the standing ask a better deal per unit; the mechanism deliberately does not reprice, because repricing on deposits would let depositors manipulate fills.

For buyers this cuts both ways. A fill just after a large distribution is the bargain case. But the reverse also holds: capital routed into a Strategy late in a nearly expired epoch is briefly exposed to being sold very cheaply - the zero-price page next makes that concrete.

FOR AUCTION BUYERS

Evaluate lot ÷ payment, not payment alone - and remember the lot can still grow between your simulation and your inclusion.

WHY THIS MATTERS

Path-dependence in the effective rate is the honest cost of refusing oracles and repricing hooks. The paper states it rather than rounding it away.

PART VI · STRATEGIES AND AUCTIONS

32 Zero-price fills and the floor restart

If nobody fills before the epoch expires, the required payment is zero - and a fill at zero is valid. This is deliberate, and it has teeth.



Accepted A-05 behaviour: the protocol prefers giving the lot away to trusting an oracle.

FIG. 20 An unfilled epoch decays to zero; the lot can then be taken for nothing. The next epoch restarts at the configured `minimumPrice` and recovers only geometrically, fill by fill.

At or past expiry, `currentPrice()` returns zero and `buy()` transfers the whole lot while collecting nothing. An acquisition Strategy gives its USDG away and receives no target token; a GBX-payment Strategy gives its USDG away and burns nothing. The next epoch then opens at `minimumPrice` - a restart floor, *not* a fill-time floor - and climbs back only by the multiplier, one fill at a time. In the worked configuration the restart is 1,000,000 raw units, essentially a fresh start from nothing.

Why tolerate this? Because the alternatives are worse: a fill-time floor needs someone to set the right floor (an oracle in disguise), and refusing zero fills leaves dead Strategies holding stranded USDG forever. The design accepts a sharply visible failure mode - free money for whoever notices - as the cost of unstoppable, oracle-free liquidation. Internal review records this as accepted behavior A-05, with two regression tests documenting the collapse and the geometric recovery.

IMPORTANT RISK

USDG routed to an ignored Strategy can be lost to a zero fill. Signalers direct capital at auctions they believe buyers will attend - that belief is part of the signal.

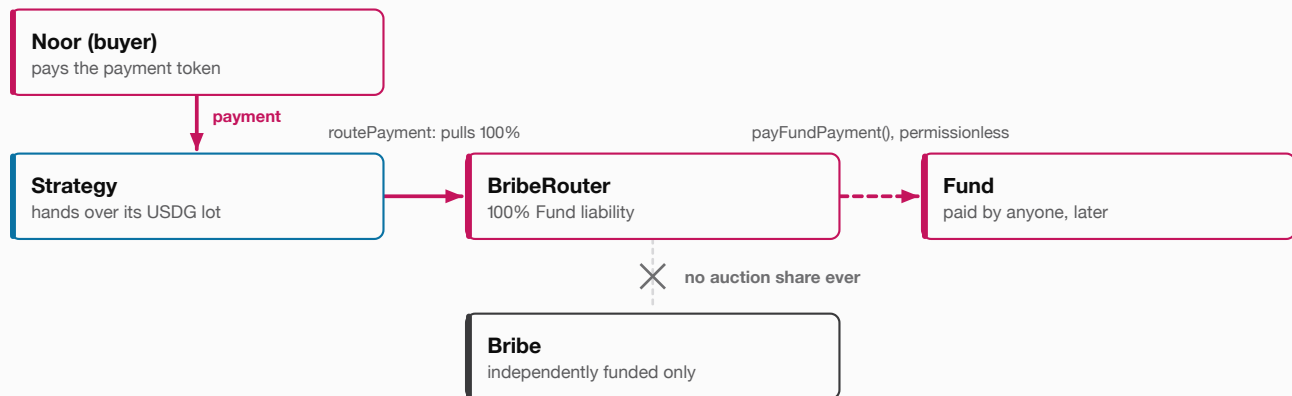
FOR AUCTION BUYERS

Watching for expiring epochs is legitimate: the mechanism explicitly pays whoever shows up. Your competition is every other watcher.

PART VI · STRATEGIES AND AUCTIONS

33 Buyer protections and settlement

Three parameters protect every fill. One rule settles every payment: all of it belongs to the Fund.



Buyer protections on every fill: expected epoch ID, a deadline, and a maximum payment. USDG leaves in the same transaction. If the payment token is GBX, Fund receives it unburned; anyone may then call `Fund.burnGBX()`.

FIG. 21 The paired router pulls the complete payment and records it as a fixed Fund liability that anyone may deliver. No share is split off anywhere - reward funding is a separate, voluntary act.

A buyer calls `buy(receiver, expectedEpochId, deadline, maximumPayment)`. The epoch check defeats being front-run into a different auction round; the deadline bounds how long a signed transaction stays valid; the payment cap is slippage protection against the lot being filled and re-opened higher mid-flight. Payment collection and lot delivery verify exact balance deltas on both legs.

Settlement is deliberately indirect: the Strategy pushes the payment into its paired **BribeRouter**, which records 100% of it as a fixed liability owed to the Fund. Anyone may call `payFundPayment()` to deliver it. The indirection preserves liveness - if the payment token temporarily rejects the Fund, auctions keep clearing and the entitlement waits, retryable, at its fixed destination. For GBX payments, delivery leaves supply untouched; the burn is a separate permissionless `Fund.burnGBX()` call, and doing it before redemptions keeps Fund-held GBX out of the redemption denominator.

An earlier design split payments ninety-to-ten with an adjustable reward share; ADR 0021 removed the split entirely. This paper documents only the shipped rule: the Fund receives everything, every time.

WHAT THE CONTRACT ENFORCES

Epoch, deadline, and cap checks; exact-transfer verification; 100% liability recording; fixed destinations no key can redirect.

FOR GBX HOLDERS

A GBX-payment fill is supply-neutral until someone burns. The burn call is free to make and benefits every remaining holder.

PART VII · STRATEGY REWARDS

34 Strategy rewards, and the word "Bribe"

Each Strategy has a companion contract that can stream tokens to its signalers. Everything about it is optional except its bounds.

The contract is named **Bribe** - a technical term inherited from established onchain voting-incentive systems, where third parties openly attach rewards to particular choices. It denotes a transparent, permissionless incentive contract, not an unlawful offchain payment; this paper says **Strategy reward** in prose and uses the contract name only where precision needs it. Whether to rename the contract remains an open product decision recorded in the internal review.

What it does: hold up to eight registered reward tokens and stream deposited amounts across the Strategy's signalers in proportion to their signal weight over time. Who funds it: anyone, voluntarily - an asset community that wants accumulation pointed its way, a partner, the curious. What never funds it: auction proceeds. Since ADR 0021 there is no settlement split, no reward percentage, and no fee of any kind; a Strategy's payments belong entirely to the Fund, and its rewards exist only if someone chooses to supply them.

Signaling therefore never guarantees a reward. It guarantees exactly one thing: if rewards are funded, they accrue to the signal weight actually present, and to nothing else.

IN ONE SENTENCE

A Strategy reward is a tip jar with rules: anyone may fill it, only that Strategy's signalers can be paid from it, and it holds at most eight kinds of token.

WHY REWARDS EXIST AT ALL

They let outsiders bid for the fund's attention in the open, on the same rails, instead of through private arrangements nobody can see.

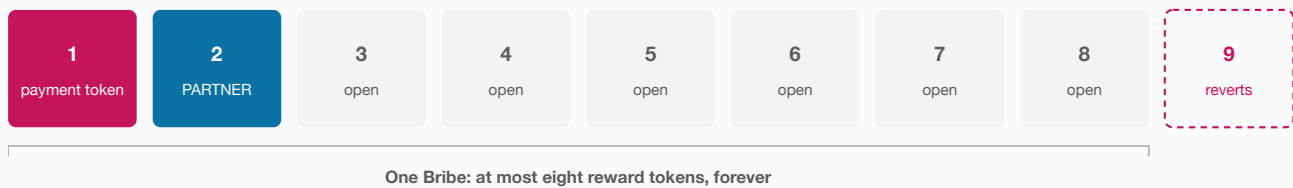
WHAT THIS DOES NOT GUARANTEE

Any yield. An unfunded reward contract pays nothing forever, and most may stay unfunded.

PART VII · STRATEGY REWARDS

35 Canonical and supplemental tokens, capped at eight

Registration is governance-gated and append-only; funding is permissionless; and the list can never exceed eight entries.



Slot 1 is automatic: the Strategy's payment token. Governance may register up to seven more. The ninth registration reverts and no one can raise the cap.

Registration schedules nothing. A token pays only what someone independently notifies.

FIG. 22 Slot one is filled automatically at Strategy creation with the payment token. Governance may register up to seven supplemental tokens; the ninth registration reverts, and the cap is immutable bytecode.

When governance creates a Strategy, the new reward contract automatically registers the Strategy's payment token as its first - *canonical* - reward token. That registration is bookkeeping, not funding: it merely makes the token eligible for streams. Governance can register further *supplemental* tokens through the timelock, each subject to the same rule: at most 8 tokens per reward contract, ever, with no removal and no cap-raising power anywhere.

Two misconceptions are worth killing early. First, a supplemental token is not something the Strategy acquires - it is just an extra currency its tip jar accepts. Second, registration by governance does not mean funding by governance: `notifyRewardAmount` is open to any address holding the token.

Why cap at eight? Every mandatory reward loop - signal entry, signal exit, the all-token claim - walks the registered list. The cap turns those walks into a bounded cost with a measured worst case (next pages) instead of an unbounded liability that a hostile registration could inflate.

WHAT THE CONTRACT ENFORCES

Append-only registration through Resonance only;
 RewardTokenLimitReached at nine;
 permissionless funding of any registered token.

FOR REWARD CLAIMANTS

The token list is public per Strategy.
 Anything not on it can never be streamed there - no matter what a frontend claims.

PART VII · STRATEGY REWARDS

36 Reward streams, exactly

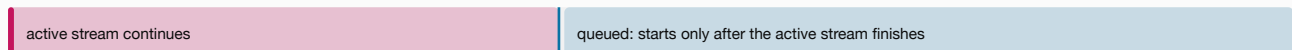
A notification starts a seven-day stream - or queues behind one - and every wei of it is eventually attributable, by construction.

Notify 350 tokens with signal present: a 7-day stream starts

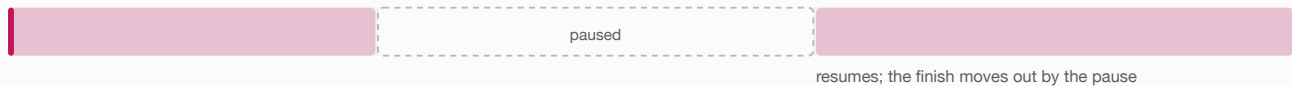


rate = $\text{floor}(350e18 / 604,800)$ per second; the remainder is emitted one wei per second at the start

Notify during a live stream: the amount queues, the live stream is untouched



All signal leaves mid-stream: time pauses; nothing is emitted to an empty room



Exact stream arithmetic from the reviewed contract: nothing rounds away, and a live stream can never be diluted by a top-up.

FIG. 23 Three honest behaviors: exact rates with the division remainder emitted first; queuing that never dilutes a live stream; and zero-supply pausing that stops the clock rather than burning the rewards.

Depositing amount A starts a stream of $\text{floor}(A / 604,800)$ per second for seven days, with the integer remainder emitted one extra wei per second over the stream's earliest seconds - so the schedule sums to A exactly. Worked: 350 wrapper tokens stream at 578,703,703,703,703 wei per second, with the first 425,600 seconds emitting one wei more.

A notification during a live stream **queues** - it never restarts or extends the active stream, which blocks the classic griefing pattern of resetting a stream with dust. If every signaler exits mid-stream, the stream **pauses**: the clock stops, nothing is emitted to an empty room, and the finish line moves out by exactly the paused duration when signal returns. Sub-unit accruals are carried per user with the same exactness the revenue index uses; unattributable carry left behind when the room empties becomes - like everything unattributable in this system - a fixed Fund liability.

This is the resolved form of finding A-03; the temporal caveat of open finding A-09 (chapter 28) applies to reward carry exactly as it does to revenue carry.

UNDER THE HOOD

Per token: an 8-term conservation identity reconciles scheduled, queued, pending, indexed, per-user carry, accrued liability, and Fund-bound amounts against the held balance.

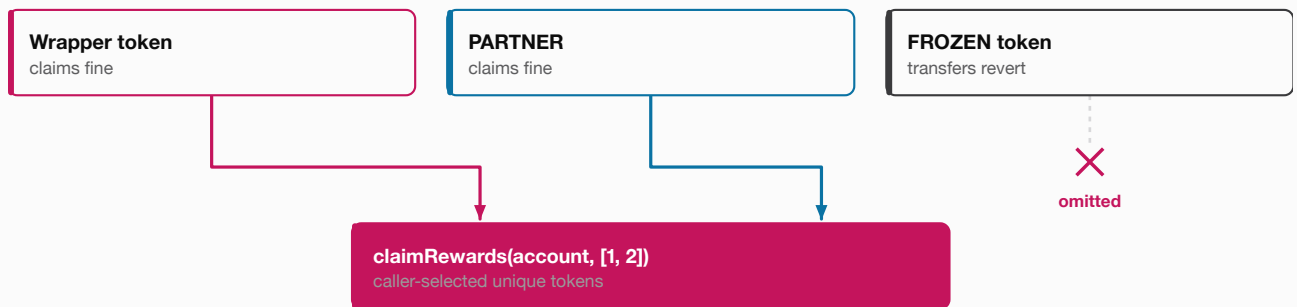
REWARD-PER-WEIGHT

Accrual is signal-weight $\times$ time: half the weight for the full week earns half the stream, all the weight for half the week earns half the stream.

PART VII · STRATEGY REWARDS

37 Selective claims, and the cost of eight

Claim one token, a chosen set, or everything. The cap keeps the worst case bounded - and measured.



Claim one token, a chosen set, or all eight - the claimant chooses.

Omitted rewards stay claimable later.

FIG. 24 A frozen or malicious registered token can block only claims that include it. Scalar and caller-selected claims route around it; omitted rewards remain claimable later.

Anyone may trigger a claim, but payment reaches only the entitled account - claiming is a favor anyone can do you, not a theft vector. The convenience path that touches all registered tokens can be blocked by one broken token; the selective paths exist precisely so that never matters.

FOR REWARD CLAIMANTS

Default to selective claims. They cost little more than the convenience path and are immune to any one token's misbehavior.

BROKEN-TOKEN ISOLATION

A failing reward token strands only its own payouts - and even those stay retryable once the token recovers. Signal exit never touches reward tokens at all.

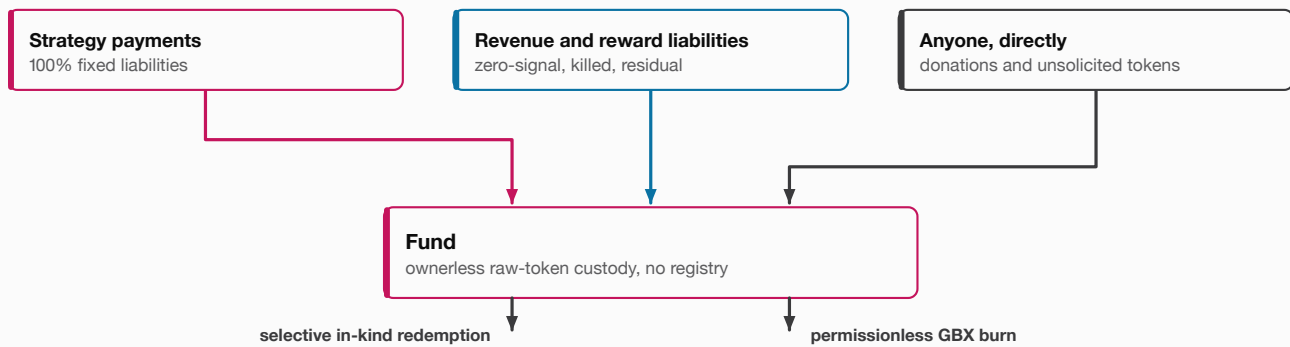
MEASURED PATH (8 REGISTERED TOKENS)	GAS
addSignal	336,621
removeSignal (worst-case exit)	1,341,818
Claim one selected token	168,113
Selective eight-token claim	1,348,052
All-token convenience claim	1,339,891
Strategy.buy	196,941
addSignal / removeSignal at one token	227,777 / 176,166

Recorded with Foundry 1.7.1, solc 0.8.26, optimizer 10,000 during internal review. Work grows roughly linearly per registered token (about 15.5k gas per token on entry, 160.5k on exit); the cap converts that slope into a ceiling - the largest required user exit stays under 1.35 million gas, comfortable margin against the documented 60-million-gas block limit. This is the "cost retained" half of finding A-08: bounded and priced, not free.

PART VIII · THE SHARED FUND

38 What the Fund is

The Fund is a vault with no keyholder: raw tokens in, redemption and burning out, and not one other door.



The only exits. Nothing else - no owner, no sweep, no migration - can move a Fund balance.

FIG. 25 Three ways in - Strategy payments, protocol liabilities, and anyone's direct transfer - and exactly two ways out, both initiated by GBX holders or the public, never by an operator.

The Fund contract is deliberately tiny. It holds whatever ERC-20 balances it holds; it can burn GBX it owns when anyone asks; and it can redeem. It has **no owner**, no upgrade path, no successor, no migration, no sweep function, and no pause - properties the repository enforces with tests that verify the removed functions do not merely revert but do not exist.

Just as deliberately, it keeps **no asset registry**. "What does the Fund hold?" is answered by reading token balances offchain, not by an onchain list. Official protocol membership - which assets the machine actively accumulates - lives in Resonance's Strategy set; the Fund itself accepts anything anyone sends it, which means unsolicited, worthless, or hostile tokens can sit there indefinitely. A registry would have to be curated, and a curator is an administrator - the thing this design refuses to have.

Direct donations simply become backing for all GBX holders. Mistaken transfers are unrecoverable, by the same immutability that protects everything else.

IN ONE SENTENCE

The Fund is shared property administered by arithmetic: nobody can take from it, and everybody with GBX can take their share of it.

WHY NO RECOVERY ADMIN

Any key that can "recover mistakes" can also make them. The design prices lost donations below the cost of a rescue power.

PART VIII · THE SHARED FUND

39 Selective in-kind redemption

Redemption pays in the assets themselves. The redeemer - not the protocol - decides which assets, and one snapshot prices them all.

PER SELECTED TOKEN

$$\text{payout} = \text{floor}(\text{fundBalanceBefore} \times \text{gbxBurned} / \text{gbxSupplyBeforeBurn})$$

One pre-burn total supply is captured first and used as the denominator for every selected token; balances are snapshotted before any movement. The GBX burn and all selected transfers then execute in one atomic transaction - if any selected transfer fails, everything reverts, including the burn.

REDEMPTION CANNOT BE PAUSED

There is no pause switch anywhere in the core - which means there is none on the exit either.

NOT "AUTOMATIC"

Redemption is a transaction you construct: your amount, your receiver, your token list. Discovery of what is worth selecting happens offchain, in your tooling.

The caller supplies the GBX amount, a receiver, and a list of unique non-GBX token addresses. Duplicates revert - detected in $O(n)$ using EIP-1153 *transient storage*, a Cancun-era feature giving contracts scratch space that lives only for one transaction, so the check needs no permanent bookkeeping. GBX itself can never be selected, and "in-kind" means exactly that: you receive the tokens, with all their properties and problems, not a cash equivalent.

Selectivity is the safety mechanism. Because the Fund can hold broken or hostile tokens, a redemption forced to touch *every* holding could be blocked by any one of them. Naming your own list makes each redemption depend only on tokens you chose - one bad asset can spoil a redemption that includes it, and no redemption that omits it.

BURN 1,000 GBX WHILE SUPPLY IS 100,000 GBX — A 1% CLAIM



FIG. 26 Selecting an asset claims a pro-rata slice of it. Omitting one forfeits that claim permanently - the value stays behind for whoever still holds GBX.

PART VIII · THE SHARED FUND

40 A worked redemption, and omitted assets

Leo burns 10,000 GBX against a 120 million supply, selects two assets, and deliberately omits a third.

LEO REDEEMS

10,000 of 120,000,000 GBX

one pre-burn supply snapshot prices every selected asset

NVDA-linked wrapper	Fund holds 5,000	pays 0.416666
USDG	Fund holds 250,000	pays 20.833333
PARTNER	Fund holds 12	omitted - forfeited to remaining holders

payout = floor(balance x 10,000 / 120,000,000) per selected token. The burn and both transfers are one atomic transaction.

FIG. 27 Every payout uses the same pre-burn denominator. The omitted PARTNER balance simply stays in the Fund, slightly enriching every remaining holder - including Leo, if he still holds GBX.

Leo's 10,000 GBX is one twelve-thousandth of the 120,000,000 pre-burn supply, so he receives exactly that fraction of each selected balance: 0.416666 wrapper tokens from the Fund's 5,000, and 20.833333 USDG from its 250,000. He omits the 12-token PARTNER dust because the claim - a millionth of a token - is worth less than the gas and the risk of touching it.

Omission is permanent for that redemption: the claim is not banked, deferred, or redeemable later at the old ratio. And flooring is real: a selected token whose pro-rata share rounds to zero (a seven-wei balance against a small burn, say) transfers nothing while still consuming a list slot. Redeemers with small burns should select accordingly.

If a selected token turns out to be frozen or malicious, the whole redemption reverts - burn included - and Leo simply resubmits without it. The failure mode is a wasted transaction, never a lost burn.

FOR REDEEMERS

Preview every balance offchain, drop dust and suspect tokens, and remember: your list, your risk, one atomic result.

MAINTENANCE FIRST

Fund-held GBX in the supply denominator dilutes every redemption. Anyone can - and should - call burnGBX and settle GBX liabilities before redeeming.

PART IX · LIQUIDITY

41 Genesis liquidity: 20 million GBX, one position

Mining economics needs a GBX price to exist. The 2% genesis allocation exists to make one - and for nothing else.

A **liquidity pool** is a standing pot of two tokens that anyone can trade against; prices move along a curve as trades tilt the pot. **Uniswap v4** is the pool system used here, and an **LP position** is a deposit into a chosen price range (delimited by *ticks*) that earns a share of trading fees while the market price is inside it. The canonical GumBall6900 pool is GBX/USDG and *hookless* - no custom code attaches to the pool, so it behaves exactly like a vanilla pool with no surprises.

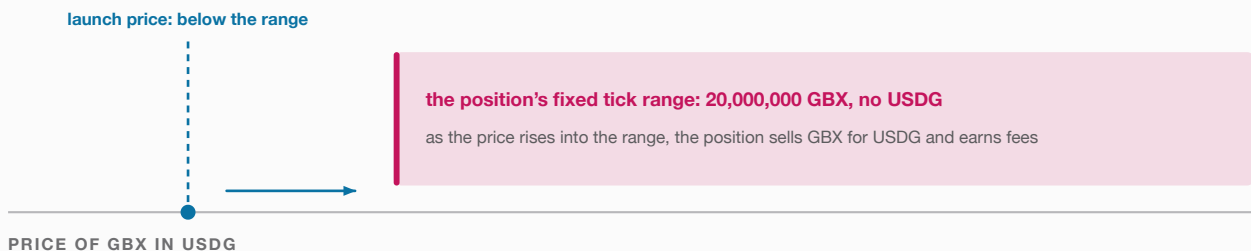
The entire 20 million GBX genesis allocation funds one precommitted position, and it launches **single-sided and out of range**: all GBX, no USDG, parked above the launch price. The protocol seeds no dollars and therefore asserts no valuation. If buyers push the price up into the range, the position sells GBX into that demand and begins earning fees; if they never do, the protocol has still promised nothing.

WHY SINGLE-SIDED

Pairing GBX with protocol-owned USDG would mean the deployer picked the launch price. Out-of-range GBX lets the market do the pricing from the first trade.

NOT A TREASURY

The 20 million is committed, not held: once the NFT is in custody, no one - not governance, not the deployer - can withdraw the principal, ever.



The protocol seeds only GBX, so the market - not the deployer - sets what a GBX is worth.

The pool is *hookless*; the range, fee tier, and position NFT are precommitted. A wrongly configured genesis position cannot be corrected afterward.

FIG. 28 The precommitted range sits above the launch price. The pool key, fee tier, tick range, and position NFT identity are all fixed before deployment and validated on delivery.

PART IX · LIQUIDITY

42 Permanent custody and one-time admission

The LiquidityPosition contract accepts exactly one NFT, checks everything about it, and then has no code path that could ever release it.

Uniswap positions are represented as NFTs, so custody means holding one token forever. At deployment the contract commits to the exact PositionManager address, depositor address, token ID, pool key hash, and tick range it will accept. The ERC-721 receiver hook validates all of them on delivery - wrong sender, wrong depositor, wrong ID, wrong pool, wrong ticks, or zero liquidity each revert - and a successful delivery flips a one-time flag. There is no transfer function, no approval function, and no owner: like the Fund, the contract has no administrator at all.

The mirror image of that safety is rigidity: an incorrectly configured genesis position - wrong fee tier, wrong range, wrong initial price - is permanent. The deployment checklist in the appendices exists mostly because of this page. Verification happens once, before delivery, or never.

WHAT THE CONTRACT ENFORCES

Six admission checks at delivery; custody queries afterward; principal invariance on every harvest. No path out exists to restrict.

IMPORTANT RISK

Irreversible genesis configuration is a real, named risk: a mispriced range cannot be corrected, only lived with.

No owner. No withdrawal. No migration. No rescue. The position is protocol anatomy, not protocol property.

PART IX · LIQUIDITY

43 Harvesting fees with fixed principal

Anyone may collect the position's accrued trading fees. The principal cannot move; the fees have exactly two fixed destinations.

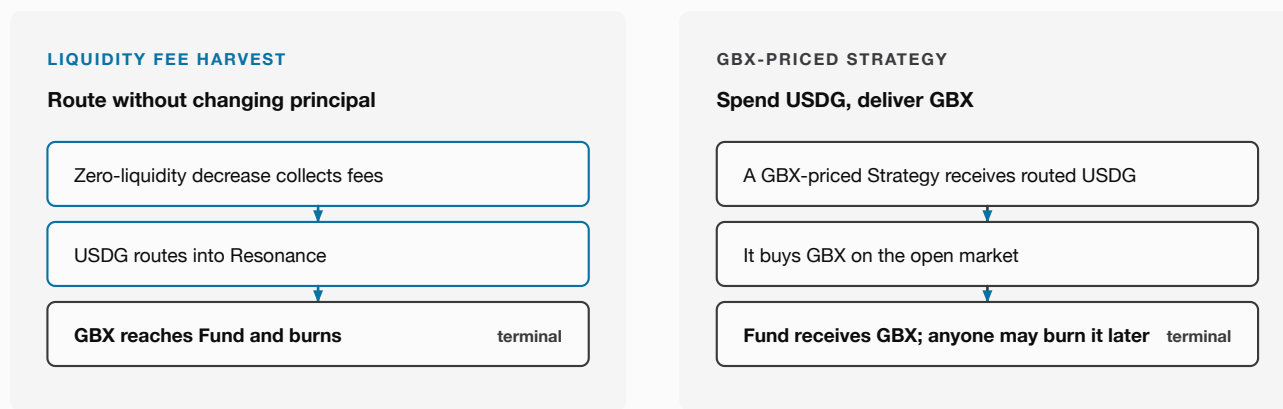


FIG. 29 `harvestFees()` collects both fee balances with a zero-liquidity decrease, verifies the principal is exactly unchanged, routes all USDG into Resonance, and sends all GBX to the Fund where it is burned - one atomic transaction.

Mechanically, the harvest uses Uniswap v4's fee-collection idiom - a liquidity decrease of zero followed by closing both currency balances - then asserts the position's liquidity equals its pre-harvest value to the unit. The collected USDG goes through `ResonanceRouter` and is allocated by the signal weights live at that moment; the collected GBX goes to the Fund and is burned in the same call. The caller funds nothing, receives nothing, and needs no permission. Direct GBX or USDG donations to the contract ride along to the same destinations on the next harvest.

Under the accepted design, liquidity fees *are* protocol revenue - USDG fees join contributions in the signal-directed path, and GBX fees shrink supply. An interim design (ADR 0018) briefly compounded fees back into the position with caller funding and a growth requirement; ADR 0022 removed all of it - the compounding surface, the `Permit2` dependency, the caller-timing question that internal review had flagged as finding A-06 - in favor of this fixed-principal routing. The integration suite exercises the shipped behavior against genuine Uniswap v4 contracts, including a 10,000-case fuzz of exact routing and a rollback test proving a failed destination restores the fee entitlement intact.

WHAT THE CONTRACT ENFORCES

Principal invariance, exact transfer deltas, fixed destinations, atomicity: a failing route or burn reverts the entire harvest, leaving fees collectible later.

FOR HARVEST CALLERS

There is no bounty. Harvest when the public benefit - revenue routed, supply burned - is worth your gas, or when you simply want the flywheel turned.

PART IX · LIQUIDITY

44 Harvest timing, honestly

Removing caller funding removed the manipulation surface internal review worried about. Two smaller timing facts remain, and they are stated here.

First: fee realization is voluntary. With no bounty, fees sit uncollected until someone pays gas. Nothing is at risk while they wait - Uniswap accounts them to the position - but revenue and burns arrive in lumps whose timing depends on volunteers.

Second: allocation snapshots at the harvest. Harvested USDG follows the signal weights present when the harvest routes it. Because signaling has no cooldown, weight can move just before a large harvest and redirect that lump. This is the same frozen current-signal policy every distribution uses, disclosed in the residual-risk record - distinct from open finding A-09, which concerns sub-index carry rather than whole distributions.

What no timing can do: touch the principal, change the destinations, split the fees, or take a cut. The earlier compound design let a sophisticated caller choose token composition at favorable pool prices; that entire class of concern ended when ADR 0022 removed caller funding. Internal review closed A-06 on that basis, and the old "A-07" question - whether LP fees should count as protocol revenue at all - was settled the same way: they do.

FIXED BY THE SHIPPED DESIGN

- Principal liquidity can never change - a harvest that would change it reverts.
- USDG fees route to Resonance; GBX fees burn through the Fund; both atomically.
- No caller funding, payout, bounty, swap, oracle, or Permit2 dependency exists.

IMPORTANT RISK

Lumpy, volunteer-timed revenue plus cooldown-free signaling means harvest-adjacent signal moves are possible and permissionless. Watch large pending fees the way you watch the timelock queue.

WHAT THIS DOES NOT GUARANTEE

That fees are meaningful. A quiet pool harvests dust; the mechanism's value scales with trading volume the protocol does not control.

LEFT OPEN, ON PURPOSE

- When harvests happen - volunteers decide.
- Which signals catch a given harvest - the live weights decide.
- Whether fees are ever large - the market decides.

PART X · GOVERNANCE MINIMIZATION

45 What signals govern, what the manager controls

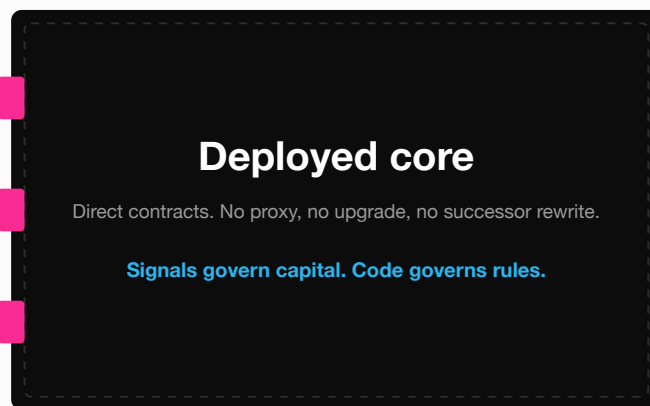
Economic direction belongs to signal holders, continuously. Human management holds three bounded powers - and the boundary is bytecode.

THREE AUTHORIZED ACTIONS

1 Add a Strategy

2 Kill a Strategy

3 Add Bribe rewards (max 8)



NO ENTRY POINT EXISTS

Proxy upgrade ⓧ

Successor migration ⓧ

Arbitrary withdrawal ⓧ

Pause switch ⓧ

General parameter setter ⓧ

Immutability removes the power to fix mistakes as surely as it removes the power to cause them. That is the trade, stated plainly.

FIG. 30 The complete ongoing management surface: add a Strategy, kill a Strategy, register a reward token within the eight-token cap. Everything on the right has no entry point at all - not a restricted one.

Resonance is the only owned core contract, and its owner - intended to be a timelock - can call exactly three ongoing functions. `addStrategy` deploys a new Strategy with its paired reward and router contracts through two bound factories. `killStrategy` permanently stops one Strategy's future revenue. `addBribeReward` registers a reward token, subject to the immutable cap of eight per Strategy. That is the entire recurring surface: there is no parameter to tune, no fee to set, no treasury to spend, and - since ADR 0021 removed the reward split and its governance setter - no percentage anywhere for management to adjust.

A handful of **one-time** setup calls exist only to wire the system at deployment: binding Resonance into SignalGBX and the two factories, binding the router into Resonance, and handing GBX minting to the Fundraiser. Each locks permanently after first use; a mistake in any of them is unrecoverable, which is why the deployment checklist verifies every one.

IN ONE SENTENCE

Management curates the menu of Strategies; it never touches the money, the weights, the schedule, or the rules.

NO FEES OF ANY KIND

No fee, no salary, no carry, no revenue share exists anywhere in production code - and no function exists through which one could be introduced.

PART X · GOVERNANCE MINIMIZATION

46 What the manager cannot do

These are not permissions withheld by policy. They are functions that do not exist in the deployed bytecode.

CANNOT

BECAUSE

Mint GBX or reopen burned capacity

Minting is locked to the Fundraiser; the ceiling counts lifetime mints

Change the emission schedule

Every constant is immutable in the Fundraiser

Withdraw or redirect Fund assets

The Fund has no owner and only redemption and burning as exits

Redirect Strategy payments or revenue

Destinations are immutable fields; liabilities are fixed at accrual

Move or unwind the liquidity position

The position contract has no owner and no outbound path

Pause anything, including redemption

No pause switch exists in any core contract

Upgrade or migrate any contract

All twelve are direct deployments; no proxy, no successor

Seize or freeze staked GBX

SignalGBX answers only to its stakers

Alter an existing Strategy's configuration

Auction parameters are immutable per Strategy

Raise the eight-token reward cap

MAX_REWARD_TOKENS is a compile-time constant

Sweep or rescue arbitrary assets

No generic call executor or sweep exists in the core

VERIFY, DON'T TRUST

Every row is checkable against the generated ABI: the functions are absent, not gated. The appendices point to the exact interfaces.

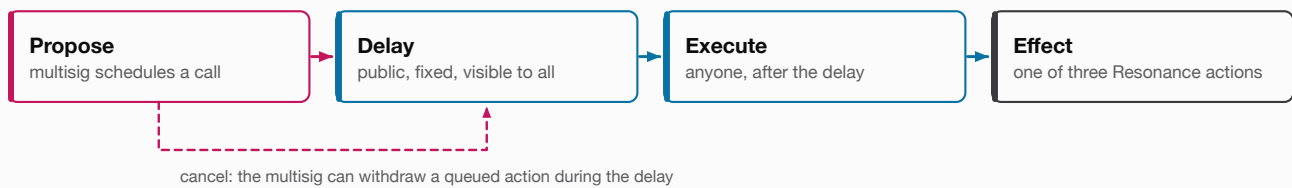
WHAT REMAINS POSSIBLE

A hostile manager could still add a junk Strategy, kill a good one, or register useless reward tokens. Bounded harm is the honest claim - not zero harm.

PART X · GOVERNANCE MINIMIZATION

47 The timelock, and a compromised key

The intended Resonance owner is a standard OpenZeppelin TimelockController: every management action crosses a public delay before it can execute.



The timelock also administers its own roles and delay - through the same public delay. Watching the queue is every user's early-warning system. Fund and LiquidityPosition take no orders from it: they are ownerless.

FIG. 31 Propose, wait publicly, execute - with cancellation available throughout the delay. Execution can be left open to anyone, so operators cannot even be a bottleneck at the finish line.

The intended configuration: the project multisig holds the proposer and canceller roles; the executor role may be open; and the controller is deployed with no external admin, administering its own roles and delay only through its own delayed operations. Be precise about what this is: the TimelockController is a generic call executor, and what bounds governance is not a calldata allowlist but the narrowness of what Resonance exposes. The accurate claim is that Resonance offers three ongoing management methods - not that only three transactions could ever pass through the timelock.

Now walk the nightmare: an attacker controls the multisig. They can propose - publicly, with the delay running - to kill Strategies, add hostile ones, or register junk reward tokens. They cannot touch anything on the previous page's list, because no function exists to call. The community's defenses are time and exit: cancel (if honest signers retain the canceller role), unwind signals, unstake, and redeem - none of which any key can pause. The worst credible outcome is disruption and misdirected future flow, not confiscation.

None of this is instantiated yet: roles, delay, and signers exist only as a deployment outline, and no signed manifest verifies them. Until one does, this chapter describes intent, and says so.

WHY IMMUTABILITY IS A TRADEOFF

The same absence of power that caps a hostile key also caps a benevolent one: bugs cannot be hotfixed, mistakes cannot be reversed, and rescue is not a phone call away. This design chooses that trade knowingly.

FOR EVERYONE

The timelock queue is the protocol's only announcement channel that cannot lie. Watch it.

PART XI · SECURITY AND TRUST

48 Security goals and layered protections

The design defends a short list of properties with several independent layers - and is explicit about the layer it still lacks.

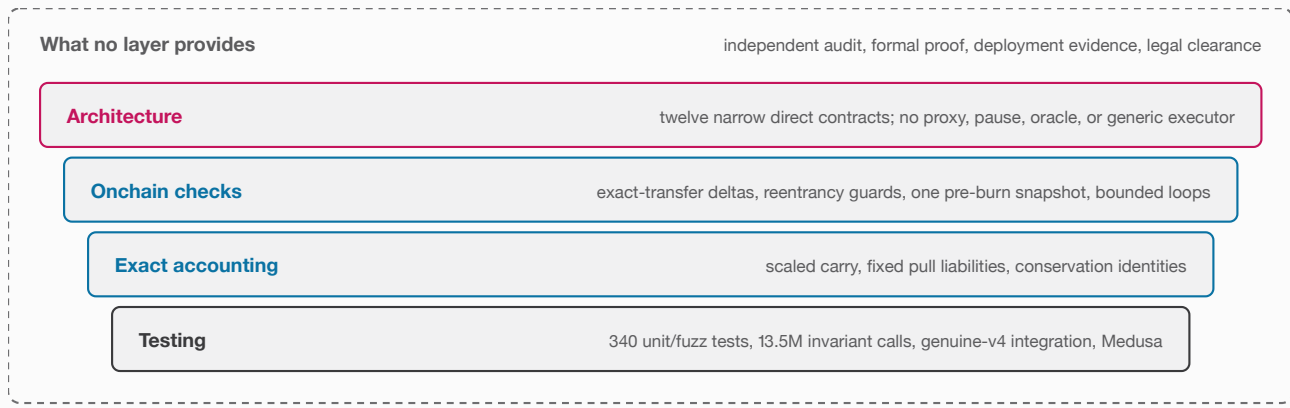


FIG. 32 Architecture, runtime checks, exact accounting, and testing reinforce each other. The dashed outer layer - independent review, deployment evidence, legal clearance - does not exist yet.

PROTECTED PROPERTY	PRINCIPAL MECHANISMS
Supply	Lifetime ceiling, locked minter, burns never reopen capacity
Contributions and settlement	Exact-transfer checks, atomic routing, buyer protections, 100% fixed liabilities
Signals and exits	Balance-bounded deltas, checkpoint-before-change, token-free exits
Solvency	Exact scaled carry with tested conservation identities everywhere
Redemption	One pre-burn snapshot, transient-storage duplicate checks, full atomicity
Custody	Ownerless Fund and position, six-check NFT admission, fixed principal

IN ONE SENTENCE

The system tries to make every failure loud, local, and retryable - and every irreversible thing deliberate.

WHAT TESTING CANNOT PROVE

Absence of evidence is not evidence of absence: no volume of passing tests demonstrates that no vulnerability exists.

Cross-cutting: every token-moving entry point is reentrancy-guarded; every supported-token transfer verifies exact deltas on both sides; and all loops are bounded by the eight-token cap or caller-chosen batch sizes.

PART XI · SECURITY AND TRUST

49 The supported-token model

The contracts assume standard, non-rebasing, exact-transfer ERC-20 behavior. Everything outside that assumption fails closed - which is protection, not support.

A *supported* token moves exactly the requested amount, keeps honest balances, does not rebase, and cannot use callbacks to re-enter guarded code. The protocol's exact-delta checks mean unsupported behavior - transfer fees, sender surcharges, rebasing, dishonest balanceOf - causes clean reverts rather than silent mis-accounting. Pauses, blacklists, and upgrades are subtler: a token that freezes the protocol's address strands its own flows (its Fund liability, its reward stream, its redemption selection) while the architecture confines the damage - fixed destinations stay retryable, selective paths route around it, and signal exits never touch it.

Each token class carries its own assumption set. **USDG** must behave for contributions, routing, and auctions to function at all - it is the load-bearing external dependency. **GBX** is protocol-authored and exact by construction. **Payment and reward tokens** are chosen by governance registration, and each one imports its issuer's whole rulebook. **Redemption selections** are chosen per call by the redeemer - the one place where token risk is entirely the user's own choice.

Tokenized stocks deserve their own paragraph. A wrapper token linked to NVIDIA or Apple stock is a claim on an issuer's arrangement - custody, redemption terms, jurisdiction - not registered ownership of shares. Issuers can freeze, blacklist, upgrade, restrict transfers to certain hours, or be compelled by regulators; the underlying may be inaccessible in your jurisdiction; and none of this is visible to, or fixable by, this protocol. Holding such a token through the Fund is holding that entire legal stack.

IMPORTANT RISK

The open A-09 finding is decimal-sensitive: for 6-decimal tokens like USDG, carry buckets can reach whole-token size at extreme signal supply. Low-decimal reward tokens deserve extra skepticism.

FOR EVERYONE

Before touching any Strategy, read its payment token like a contract - because it is one, written by someone else, changeable by someone else.

PART XI · SECURITY AND TRUST

50 The dependency map

A protocol is never more trustworthy than the things it stands on. These are all of them.

Robinhood Chain Cancun/EIP-1153 execution; verified read-only at a pinned block	Solidity 0.8.26 compiler correctness; two known upstream bugs reviewed as not applicable	OpenZeppelin 5.6.1 ERC-20, Permit, Votes, ReentrancyGuard, TimelockController	Uniswap v4 (pinned) PoolManager and PositionManager for the canonical position
USDG issuer-controlled stablecoin; 6 decimals; freeze and upgrade powers external	Payment tokens each wrapper's issuer, freeze, and legal terms are external	Reward tokens up to eight per Bribe; each fails independently	Wallets and RPC what you sign and what you see; outside the protocol
Frontend and subgraph presentation only; verify against the chain	Multisig signers propose and cancel through the timelock	TimelockController the only owner of Resonance	Asset discovery Fund holdings are found offchain; no onchain registry exists

FIG. 33 Pinned versions where pinning is possible: OpenZeppelin 5.6.1, Uniswap v4 core 1.0.2 and periphery 1.0.3, Solidity 0.8.26. Chain-level facts were checked read-only at Robinhood Chain block 32,035,314.

Three dependency notes matter most. **The chain:** Fund redemption requires EIP-1153 transient storage; internal review executed TSTORE/TLOAD successfully in an `eth_call` at the pinned block and recorded the documented Uniswap addresses and code hashes. **The libraries:** OpenZeppelin supplies the token standards, guards, and the TimelockController; Uniswap supplies the pool machinery; both are pinned, and the two known Solidity 0.8.26 compiler bugs were reviewed as not applicable to this build configuration. **The humans:** multisig signers, volunteer maintenance callers, and every token issuer remain outside any pin.

PRESENTATION IS NOT PROTOCOL

The frontend and subgraph can be wrong, censored, or offline without affecting a single onchain rule. Anything that matters can be done and verified against the chain directly.

PART XI · SECURITY AND TRUST

51 What was actually tested

Every number below is a recorded internal-engineering result. Blocked and invalid runs are listed with the same prominence as passes.

CAMPAIGN	TOOLING	RECORDED RESULT
Unit and fuzz tests	Foundry 1.7.1, solc 0.8.26	340/340 pass; 27 fuzz properties × 10,000 runs. Re-run for this edition: 340/340.
Stateful invariants	Foundry invariant engine	27 properties × 1,000 runs × depth 500 = 13,500,000 calls, zero reverts, 22-action reachability
Uniswap v4 integration	Genuine pinned v4 contracts	17/17 pass; harvest exactness fuzzed 10,000 cases
Compiler parity / supply	Hardhat 2.29.0	2/2 pass
Independent state fuzzing	Medusa 1.5.1	101,840 calls, 3,632 branches, 62/62 pass (23 properties, 39 assertions)
Second fuzzer	Echidna	INVALID: pinned 2.3.2 blocked (no Docker); native 2.3.3 crashed at start (0/25). Release blocker.
Coverage (reduced profile)	forge coverage	92.98% lines, 92.86% statements, 80.22% branches, 89.34% functions
Static analysis	Slither, Aderyn, Semgrep, Solhint	186 findings dispositioned in 23 classes; Semgrep 0; 6 secret-scan matches await classification
Mutation testing	None pinned	NO current-tree score exists; historical figures disowned. Release blocker.
Symbolic / formal	Mythril 0.24.8	Fails closed on Cancun opcodes. No formal proof exists.
Fork validation	JSON-RPC, read-only	No current-graph fork ran. Chain 4663, block 32,035,314: EIP-1153 verified, v4 code hashes recorded.
Nightly deep profile	Foundry nightly config	Configured (100k fuzz; 10k invariant runs, depth 1,000) but never completed - not counted.

Commit precision, stated once more: the register names candidate 54e3f2c...; production Solidity then changed (checked increments in the Fundraiser; the ADR 0022 harvest redesign) and the finalized campaign above was recorded against the resulting tree, committed as 5ad1ebc.... The exact final commit therefore has recorded evidence but no separately signed re-review - a distinction a careful reader deserves.

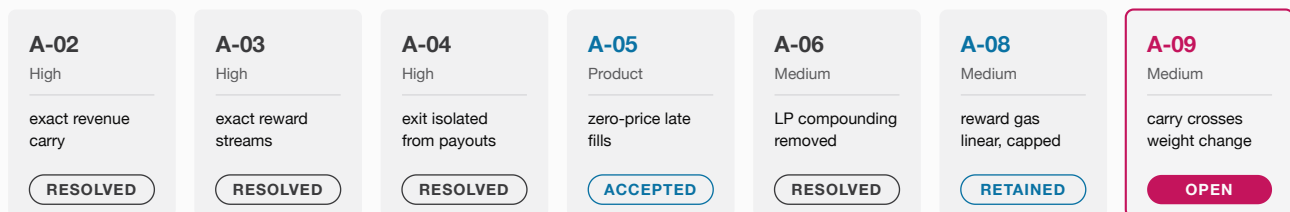
NOT "AUDITED"

These are internal results on internal infrastructure. Independent external audit: not completed. The word "audited" does not describe this system.

PART XI · SECURITY AND TRUST

52 The finding register

Internal adversarial review tracked its findings by identifier. Here is each one's final recorded state - none omitted.



From the internal register. No unresolved Critical or High finding is known internally; A-09 remains an open Medium allocation-fairness finding.

An independent external audit has not been performed. This board is internal evidence only.

FIG. 34 Three High findings resolved by redesign; one Medium resolved by removal; one accepted with disclosure; one retained as a bounded cost; one open pending an owner and independent-reviewer decision.

ID	SEVERITY	STATE	ONE-LINE SUBSTANCE
A-02	High	Resolved in the reviewed candidate	Exact carried USDG revenue accounting replaces lossy index flooring.
A-03	High	Resolved in the reviewed candidate	Bribe keeps exact rate and index carry, zero-supply pausing, queues, and selective claims.
A-04	High	Resolved in the reviewed candidate	Fixed Fund liabilities isolate signal exit from failing destination transfers.
A-05	Product behavior	Accepted and disclosed	A late fill can clear at or near zero and the next auction restarts from its configured floor.
A-06	Medium	Resolved by ADR 0022	Caller-funded LP compounding was removed; harvesting preserves principal with fixed destinations.
A-08	Medium	Liveness resolved; linear cost retained	Reward loops stay capped at eight tokens with scalar and selective escape paths.
A-09	Medium	Open; owner decision required	Conserved sub-index carry can be shared with signal weight that arrives after the value did.

WHERE EACH IS EXPLAINED

A-02 and A-09 in Part V; A-03 and A-08 in Part VII; A-04 in Part IV; A-05 in Part VI; A-06 in Part IX. Every disclosure lives beside its mechanism.

REMAINING RELEASE BLOCKERS

Independent audit; A-09 resolution or explicit acceptance; a real mutation campaign; a working second fuzzer; symbolic checks; secret-scan classification; legal and provenance clearance; signed deployment evidence.

PART XI · SECURITY AND TRUST

53 What the protocol cannot guarantee

A short chapter, on purpose. Everything here is outside any contract's reach, no matter how well the code works.

-
- 01 That GBX has, holds, or ever reaches any particular value - there is no peg, no NAV, no floor.
 - 02 That mining is profitable on any day, or that miners participate at all.
 - 03 That any auction clears, clears well, or clears before reaching zero.
 - 04 That signaling earns anything - reward funding is voluntary and may never happen.
 - 05 That Fund assets keep value, stay transferable, or remain legally accessible - each answers to its issuer.
 - 06 That redemption yields anything worth having: it is a share of what is actually there, in kind.
 - 07 That volunteers perform unpaid maintenance promptly - settlement, distribution, harvesting, and burning all wait for someone.
 - 08 That immutable code is correct code: no amount of internal testing proves the absence of vulnerabilities, and what ships cannot be fixed.

IN ONE SENTENCE

The contracts constrain behavior, not outcomes: they can make the game fair and its rules permanent - never make it a good game to play.

The affirmative guarantees this paper does make - supply ceilings, routing, settlement, exit liveness, atomic redemption, custody - are collected with their exact mechanisms in the appendices, each one stated as a testable property rather than an assurance.

PART XII · USING THE PROTOCOL

54 Walkthroughs: mine, stake, signal

Assuming a wallet, gas, and verified contract addresses. Every step is a transaction you sign; nothing happens on your behalf.

Contribute and claim (miner)

- 01 Verify the Fundraiser address independently; approve it - or rather, approve nothing; contributions pull USDG via allowance to the router path, so set your USDG allowance for the Fundraiser first.
- 02 Check the current epoch's scheduled emission and running total contributed - your eventual cost is $\text{total} \div \text{emission}$, and the day is not over.
- 03 Call `contribute(beneficiary, amount)` - minimum 0.01 USDG; the beneficiary can be you or anyone.
- 04 After the epoch ends, ensure it is settled (anyone can call `settleEpochs`), then `claim(account, epoch)`. GBX mints straight to the beneficiary; claims never expire.

Stake and signal (signaler)

- 01 Approve `SignalGBX` for your GBX, then `stake(amount)` - one sGBX per GBX, instantly.
- 02 Pick live Strategies (check `isStrategyAlive`) and `addSignal(strategy, amount)` each, or batch with `addSignalMany`. Amounts are deltas from your unallocated balance.
- 03 Adjust freely: `removeSignal` frees weight instantly; `unstake(amount)` returns GBX for anything unallocated.
- 04 After a Strategy is killed: `removeSignal` still works - exits never depend on any token transfer.

FOR CONTRIBUTORS

Your USDG is working - and at risk - from the moment it routes, which is the same transaction. There is no cancel, no refund, and no cooling-off.

GAS EXPECTATIONS

Signal operations walk the Strategy's reward-token list: entry from ~228k gas (one token) to ~337k (eight); exit up to ~1.34M at the cap. Budget accordingly.

PART XII · USING THE PROTOCOL

55 Walkthroughs: buy, claim, redeem, maintain

Auction fills, reward claims, redemptions, and the unpaid maintenance calls anyone can make.

Fill an auction (buyer)

- 01 Read `availableRevenue()` (the lot), `currentPrice()` (the full-lot ask), and `epochId`. Judge `lot ÷ price`, not price alone.
- 02 Approve the Strategy for the payment token, then `buy(receiver, expectedEpochId, deadline, maximumPayment)`. All three protections are yours to set tightly.
- 03 You receive the USDG immediately; the payment becomes a Fund liability anyone may deliver via the router's `payFundPayment()`.

Claim rewards (signaler)

- 01 Check `earned(account, token)` per registered token on the Strategy's Bribe.
- 02 Prefer `claimReward(account, token)` or the selective `claimRewards(account, tokens[])` - immune to any one broken token.

Redeem (holder)

- 01 Inventory Fund balances offchain; drop dust and tokens you distrust.
- 02 First do - or wait for - the free hygiene calls: settle GBX liabilities, then `Fund.burnGBX(pendingGBX())`; Fund-held GBX in the denominator dilutes you.
- 03 Approve the Fund for your GBX and `redeem(amount, receiver, tokens[])`. One atomic transaction: your burn and every selected payout, or nothing.

Maintain (anyone)

- 01 `settleEpochs · distributeAll / distributeRange · indexPendingRevenue · syncRevenue · payFundRevenue · payFundPayment · payFundReward · harvestFees · burnGBX` - all permissionless, all unpaid, all useful.

FOR AUCTION BUYERS

Set `maximumPayment` barely above the current ask and a tight deadline. The epoch check already protects you from filling a re-opened, repriced round.

FOR REDEEMERS

Omitted is forfeited. A reverted redemption costs gas but returns your GBX - the burn only survives if every selected payout does.

PART XII · USING THE PROTOCOL

56 Safety checklists

Seven roles, seven short lists. If a line fails, stop - the protocol will still be there tomorrow.

ROLE	CHECK BEFORE ACTING
Everyone	Contract addresses verified from more than one independent source · this deployment actually exists and matches reviewed code (none does today) · wallet on the right chain · you can afford total loss
Contributor / miner	Scheduled emission and running total for today · realistic exit liquidity for GBX · allowance no larger than needed · you accept that the day's final cost is unknown until it ends
Signaler	Strategy is alive · you understand its payment token and issuer powers · reward-token list is what you expect (≤ 8) · exit gas at the current token count is acceptable
Auction buyer	Lot ÷ ask beats your own valuation with margin · expectedEpochId is current · deadline tight · maximumPayment tight · payment-token allowance exact
Reward claimant	earned() is worth the gas · claiming selectively · a claim triggered by someone else still only pays you
Redeemer	Every selected token previewed and transferable · dust dropped · Fund-held GBX burned first · payout after flooring is still worth it · receiver address triple-checked
Maintenance caller	The call actually has something to move · gas cost accepted as a gift to everyone · no assumption anyone will thank you
Multisig signer	Proposal payload decoded and understood · targets one of the three real actions · community notice given · cancellation path rehearsed · keys stored like the protocol depends on them, because it does

The protocol cannot be paused, so it also cannot wait for you to be careless. **Verify, then sign.**

PART XIII · QUESTIONS, ANSWERED

57 FAQ: the idea, mining, and signaling

Short answers with chapter-length backing. Part and chapter references throughout the paper carry the detail.

Is GumBall6900 a traditional index fund?

No. There is no methodology, no committee, no rebalancing, and no NAV. It is a mechanism for forming a basket over time out of signal-directed purchases.

Is GBX pegged to the Fund's value?

No. Nothing computes or defends a price. GBX's redemption right is a pro-rata in-kind claim on selected Fund balances - whatever those happen to be worth.

Does the Fund have a fixed target basket?

No. Composition is the accumulated history of past acquisitions plus anything sent to it, minus redemptions. It never snaps to current preferences.

Who chooses future acquisitions?

sGBX signalers, continuously, within the menu of Strategies governance has created. Weight in equals share of future inflow out.

Can signals sell assets the Fund already holds?

Never. Signals steer inflow only. The only way assets leave the Fund is redemption (and GBX burning).

How do I mine GBX?

Send USDG (minimum 0.01) to the Fundraiser during an epoch, then claim your pro-rata share of that day's fixed emission after it settles.

Is mining profitable?

Sometimes, for some participants, never guaranteed. Your cost is set by everyone else's contributions; your revenue by a market this protocol does not control.

What happens in an empty epoch?

Its scheduled emission is forfeited forever. The schedule advances on wall-clock time regardless of attendance.

Can more than one billion GBX ever exist?

No. Lifetime minting is capped at exactly 1,000,000,000, checked on every mint.

Do burns let more GBX be minted?

No. The ceiling counts lifetime mints. Burns shrink supply and re-fill nothing.

What is sGBX?

A non-transferable receipt minted 1:1 for staked GBX. It measures signal capacity and nothing else.

Can sGBX be transferred or sold?

No. Any transfer that is not a mint or burn reverts. Influence stays attached to exposure.

Must I allocate all my sGBX?

No. Idle sGBX is fine; it earns nothing, dilutes nobody, and unstakes instantly.

Are signals percentages?

No. The chain stores absolute per-Strategy amounts. A frontend may display percentages, but you submit amounts.

Does adding a Strategy resize my existing signals?

No. Every operation is an independent delta against one Strategy. Nothing else in your account moves.

PART XIII · QUESTIONS, ANSWERED

58 FAQ: revenue, auctions, and rewards

The middle of the machine: how money moves, clears, and optionally tips.

Do I ever have to reset my whole account?

No such operation exists. The old whole-account model was removed; only per-Strategy deltas remain.

What happens after a Strategy is killed?

It stops receiving future USDG; its undistributed revenue becomes a Fund liability. Your signal on it stops mattering and can be removed whenever you like.

Can a frozen token trap my signal?

No. Signal removal and unstaking transfer no tokens at all, so no token's behavior can block them.

How does USDG reach Strategies?

One path: contributions and harvested fees enter ResonanceRouter, Resonance indexes them by live signal weight, and anyone triggers distribution.

What happens to tiny USDG amounts?

They wait as exact carry until enough accumulates to move the index - nothing is rounded away. See also the open A-09 caveat about who shares carry across weight changes.

What does a Strategy auction actually sell?

Its entire current USDG balance, as one lot, for a payment in its configured token that declines with time.

Why no price oracle?

Oracles are trusted reporters and trusted reporters fail adversarially. The falling-clock design discovers price from actual buyer behavior instead.

Is the quoted auction number a per-USDG price?

No - it is the payment for the whole lot, however large the lot has grown during the epoch. $\text{Judge lot} \div \text{payment}$.

Can the auction really reach zero?

Yes. At expiry the lot can be taken for nothing, and the next epoch restarts at its configured floor. Accepted, disclosed behavior (A-05).

What share of a payment reaches the Fund?

All of it. 100%, recorded as a fixed liability at fill time. No split, no fee, no reward share - and no function to create one.

Can management change that percentage?

No. The ninety-ten split of an earlier draft and its setter were removed from the design entirely; the Fund receiving everything is not a parameter.

What is a Bribe?

The technical name of each Strategy's reward contract - an open incentive pot inherited from onchain voting systems. This paper says "Strategy reward". It implies no offchain payment.

Can one Strategy have several reward tokens?

Yes - up to eight: its payment token automatically, plus up to seven more that governance registers.

How do supplemental reward tokens get funded?

Anyone deposits via `notifyRewardAmount`. Registration only makes a token eligible; funding is always a voluntary external act.

Why an eight-token limit? Can governance raise it?

It bounds every mandatory reward loop, keeping worst-case exit gas under $\sim 1.35\text{M}$. The cap is a compile-time constant; nobody can raise it.

PART XIII · QUESTIONS, ANSWERED

59 FAQ: the Fund, governance, and status

Exits, powers, and the honest answers about where this project actually stands.

Can a broken reward token block my claims?

Only claims that include it. Claim tokens individually or as a chosen set and it cannot touch you.

Does signaling guarantee rewards?

No. Unfunded reward contracts pay nothing, and most may stay unfunded. Signaling guarantees only your share of whatever is actually funded.

What is a buyback Strategy?

A Strategy whose payment token is GBX: buyers pay GBX for its USDG, the GBX lands in the Fund, and anyone may burn it there.

Where does burned GBX go?

Out of existence. Supply falls, lifetime-burned rises, and the mint ceiling is unaffected.

Does the Fund keep an asset list?

No. Holdings are discovered by reading balances offchain. Strategy membership - what the protocol actively accumulates - lives in Resonance.

What if I omit an asset when redeeming?

That claim is forfeited; the value stays for remaining holders. Omission is the intended tool for skipping broken or worthless tokens.

Can one bad token block all redemptions?

No - only redemptions that select it. Your list, your dependencies.

Can management withdraw Fund assets?

No function exists for anyone - manager included - to move Fund assets except redemption and GBX burning.

Can the contracts be upgraded or paused?

No. All twelve are direct, unproxied deployments with no pause switch. What ships is final, bugs included.

Can the liquidity NFT be pulled out?

No. The custody contract has no owner and no outbound transfer path. Delivery is one-way.

Why would anyone harvest fees or run maintenance?

No bounty exists. Beneficiaries - holders wanting burns, signalers wanting routed revenue - have reasons; the design accepts volunteer timing.

Is every amount a harvest moves a "fee"?

Mostly, plus any direct donations to the position contract. Either way it takes the same fixed route: USDG to Resonance, GBX to the burn.

Is GumBall6900 a registered investment fund?

No, and this paper is not an offer of one. Securities, collective-investment, and other laws vary by jurisdiction and legal review has not been performed.

Are tokenized stocks the same as owning shares?

No. They are issuer-dependent claims with their own freeze, upgrade, transfer, and legal limitations - none of which this protocol can see or fix.

Has GumBall6900 been audited?

No independent external audit has been completed. Internal adversarial review exists, with one open Medium finding (A-09), and its full evidence is in Part XI.

What should I verify before interacting?

That a deployment exists at all (none does today), that its addresses and bytecode match reviewed code, that the timelock roles are as documented - and every line of the checklist page.

PART XIV · CONCLUSION

60 The worked example, end to end

Every thread from the paper, tied once: eight steps, four fictional people, and numbers the build recomputes on every run.

1	Maya contributes 1,000 USDG on day 121; the epoch emits 439,393.27 GBX	claims 10,984.83 GBX
2	stakes 1,500 GBX; signals 900 to NVDA, 400 to AAPL; unstakes the idle 200	absolute deltas
3	12,345.678901 USDG routes at live weights 50 / 33.3 / 16.7%	6,172.83 to NVDA
4	Noor fills the NVDA auction 17h in: pays 17.5 wrapper units for 6,172.83 USDG	next epoch opens at 35
5	100% of the payment becomes a Fund liability; a keeper pays it in	Bribes get nothing automatic
6	an independent funder streams 350 wrapper tokens over 7 days across NVDA signalers	Elena claims her share
7	the GBX-payment Strategy fills; 30,000 GBX reaches Fund; anyone burns it	supply falls
8	Leo burns 10,000 GBX of 120,000,000: takes the wrapper and USDG, omits PARTNER	0.4166 wrapper + 20.83 USDG

All participants fictional; all amounts computed by the worked model from the reviewed rules. Nothing here is a projection.

FIG. 35 One pass through the whole machine. Two scenarios worth replaying against it: a frozen wrapper token would strand only its own payouts and claims (steps 5-6) while every exit stayed live; and a signal shift just before step 3 or a fee harvest would legally redirect that lump - the disclosed timing surface of a cooldown-free system.

The example is deliberately unglamorous. Nobody in it got rich; a fifth of a reward stream and a twelve-thousandth of a treasury are the kind of numbers real mechanisms produce. What the thread demonstrates is narrower and more valuable: at every step, the rule that applied was published in advance, applied to everyone identically, and executed without anyone's permission - and every number in it came out of the same arithmetic the contracts run.

ALL FICTIONAL

The people are inventions; the tokenized assets are illustrative; availability of any real wrapped asset is not implied; nothing here is a projection of returns.

PART XIV · CONCLUSION

61 What GumBall6900 makes possible

Not a prediction of success - a statement of what this mechanism, if deployed and sound, would let people do that they cannot easily do today.

It would let strangers form a shared portfolio without trusting a manager, by making the only discretionary act - choosing what to accumulate next - continuous, proportional, and public. It would let them leave without permission, in kind, at arithmetic's price rather than an operator's. And it would make the whole arrangement inspectable: every rule cited in this paper is a line of immutable code, every number a recomputable fact.

- 1 GBX distribution is fully public: a fixed, self-funding schedule with no insider allocation of any kind.
- 2 Direction is continuous and granular: absolute per-Strategy signals, adjustable at any moment, with idle weight instantly liquid.
- 3 Acquisition needs no oracle: falling-clock auctions discover prices from real buyers, and accept the visible cost of zero-price expiry.
- 4 Settlement is total: every Strategy payment belongs to the Fund - no splits, no fees, no exceptions.
- 5 Rewards are honest: optional, externally funded, capped at eight tokens, and never a tax on shared backing.
- 6 Exit is unstoppable: selective in-kind redemption with one snapshot, full atomicity, and no pause switch anywhere.
- 7 Management is three bounded actions behind a public delay - and the Fund and liquidity position answer to no one at all.
- 8 Status is stated plainly: internally reviewed with one open Medium finding, unaudited externally, undeployed, and unlicensed for user funds until every recorded blocker clears.

THE NARROW CLAIM

Not that holders will choose well - only that the choosing can be public, continuous, and inspectable, with exits no one can close.

WHAT WOULD CHANGE OUR MINDS

Independent review finding the mechanism unsound, A-09 proving unacceptable at real scale, or legal review foreclosing the design. Each would be reported the way this paper reports everything else.

Public mining. Continuous signals. Market-executed acquisitions. Total settlement. In-kind exit. A fixed core - stated honestly, evidence and open questions included.

APPENDIX A

Contract map

The complete production surface: twelve direct, non-upgradeable contracts plus the standard timelock that owns exactly one of them.

CONTRACT	ROLE	OWNER	KEY ENTRY POINTS
GBX	Capped token; permit, votes	None (minter locked)	mint · burn · setMinter ¹
Fundraiser	Daily USDG-for-GBX mining	None	contribute · claim · settleEpochs
SignalGBX	1:1 non-transferable receipt	Deployer ¹	stake · unstake · setResonance ¹
ResonanceRouter	Single revenue entrance	None	route
Resonance	Signals and allocation	Timelock (intended)	signal ops · distribute* · sync/index · payFundRevenue · addStrategy · killStrategy · addBribeReward
StrategyFactory	Deploys Strategy pairs	Deployer ¹	createStrategy (Resonance only)
Strategy (each)	Reverse Dutch auction	None; config immutable	buy · currentPrice
BribeFactory	Deploys Bribes	Deployer ¹	createBribe (Resonance only)
BribeRouter (each)	100% payment liabilities	None	routePayment · payFundPayment
Bribe (each)	Capped 8-token streams	None	notifyRewardAmount · claims · payFundReward
Fund	In-kind redemption treasury	None	redeem · burnGBX
LiquidityPosition	NFT custody, fee routing	None	harvestFees · onERC721Received ¹
TimelockController	OZ standard; owns Resonance	Self-administered	schedule · execute · cancel

¹ one-time. Dependencies: OpenZeppelin 5.6.1; Uniswap v4 core 1.0.2 / periphery 1.0.3. Permit2 appears only in test tooling. Interfaces: IBribe, ICoreResonance, IFund, IResonanceRouter. Generated documentation for all 252 public ABI functions: [docs/reference/contracts.md](#).

APPENDIX B

Exact formulas

All arithmetic is integer arithmetic. `floor()` is written explicitly wherever the EVM floors.

RULE	FORMULA
Lifetime supply	$\text{lifetimeMinted} \leq 1,000,000,000e18$; $\text{remaining} = \text{ceiling} - \text{lifetimeMinted}$; $\text{supply} = \text{lifetimeMinted} - \text{lifetimeBurned}$
Emission step	$\text{scheduled}_0 = 465,152,749,681,042,811,702,004 \text{ wei}$; $\text{scheduled}_{t+1} = \text{floor}(\text{scheduled}_t \times 999,525,354,337,060,160 / 10^{18})$
Schedule end	99,884 nonzero epochs; $\Sigma = 979,999,999,999,999,181,815,005,172 \text{ wei}$; $\text{remainder} = 818,184,994,828 \text{ wei}$
Contributor claim	$\text{reward} = \text{floor}(\text{contribution} \times \text{epochEmission} / \text{totalEpochContributions})$
Signal capacity	$\text{available} = \text{sGBX.balanceOf}(\text{account}) - \text{accountSignalWeight}[\text{account}]$; every add/remove is a delta against one Strategy
Revenue indexing	$\text{pendingScaled} += \text{amount} \times 10^{18}$; $\text{indexDelta} = \text{floor}(\text{pendingScaled} / \text{totalSignalWeight})$; $\text{index} += \text{indexDelta}$; $\text{pendingScaled} -= \text{indexDelta} \times \text{totalSignalWeight}$
Strategy accrual	$\text{accruedScaled} = \text{remainder} + \text{weight} \times (\text{index} - \text{lastIndex})$; $\text{claim} += \text{floor}(\text{accruedScaled} / 10^{18})$; $\text{remainder} = \text{accruedScaled} \bmod 10^{18}$
Auction price	$\text{payment}(t) = \text{initial} - \text{floor}(\text{initial} \times \text{elapsed} / \text{duration})$, zero at/after expiry
Next opening	$\text{next} = \text{floor}(\text{payment} \times \text{multiplier} / 10^{18})$, floored at <code>minimumPrice</code> , capped at $2^{192} - 1$
Settlement	$\text{fundLiability} += \text{payment}$ (the whole payment, every fill)
Stream rate	$\text{rate} = \text{floor}(\text{amount} / 604,800)$ per second, +1 wei/second for the first ($\text{amount} \bmod 604,800$) seconds
Reward accrual	same index-and-remainder shape as revenue, per token, over Bribe virtual balances
Redemption payout	$\text{payout} = \text{floor}(\text{fundBalanceBefore} \times \text{gbxBurned} / \text{gbxSupplyBeforeBurn})$, one pre-burn supply snapshot for all selected tokens
Harvest invariant	$\text{positionLiquidityAfter} = \text{positionLiquidityBefore}$; contract USDG and GBX balances end at zero

Bounds fixed at deployment: auction epochs in [1 hour, 365 days]; multipliers in $[1.1\times, 3.0\times]$; configured prices in $[10^6, 2^{192}-1]$; minimum contribution 10,000 raw USDG units; reward tokens ≤ 8 per Bribe; index precision 10^{18} throughout.

VERIFIED HOW

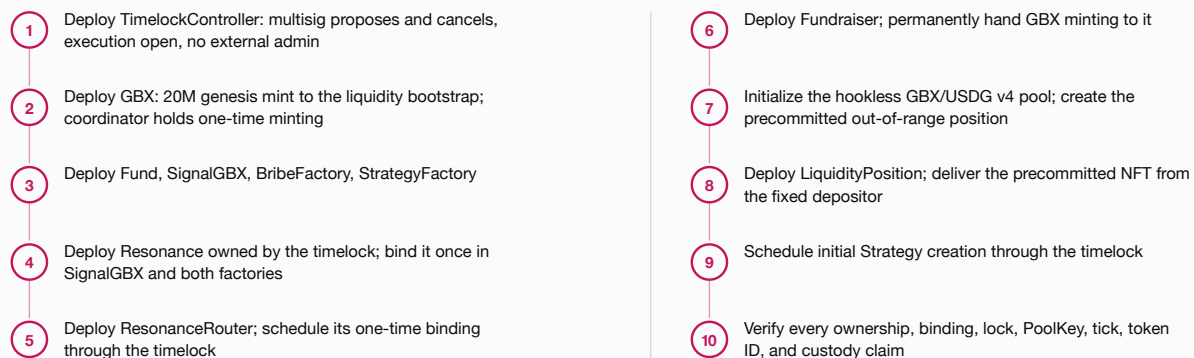
The build replays the emission schedule and stream arithmetic and cross-checks the repository's tested fixtures before a page renders (protocol-facts.mjs).

APPENDIX C

Lifecycle summaries

Each mechanism's states and transitions, compressed to one row each.

LIFECYCLE	PATH
Fundraiser epoch	open (day t) → ended → settled in strict order (anyone, batched) → claims minted per account, forever
Staking	GBX → stake → sGBX ($\pm$ signals) → unstake unallocated → GBX
Signal	add (live Strategy only) ↔ remove (always, even after kill) → zero removes list entry via swap-and-pop
Revenue	router → notify → carry/index → checkpoint on touch → distribute (live) or Fund liability (zero-weight, killed) → payFundRevenue
Auction epoch	open at price P → linear decay → fill (protected) or expiry at 0 → next epoch at $\max(\text{payment} \times m, \text{floor})$
Payment	fill → router pulls 100% → fixed Fund liability → payFundPayment (anyone) → Fund [→ burnGBX if GBX]
Reward stream	notify → start (signal present, none active) / queue → 7-day exact stream $\pm$ zero-supply pause → claims; empty-room carry → Fund liability
Redemption	select unique non-GBX tokens → snapshot supply and balances → pull and burn GBX → pay each selected → all-or-nothing
LP admission	precommitted NFT from fixed depositor → six checks → recorded forever
LP harvest	zero-liquidity decrease → principal unchanged → USDG routed, GBX burned
Strategy life	created (immutable config) → live → killed (signals removable, revenue to Fund)
Deployment	the figure below; every binding one-time; nothing executed to date



An unexecuted outline from docs/DEPLOYMENT.md. No signed manifest exists; no script is authorized to broadcast.

FIG. C.1 The intended deployment order. Unexecuted: no signed manifest exists and no script is authorized to broadcast.

APPENDIX D

Access-control matrix

Who may call what, what moves, and whether it can ever be undone.

FUNCTION	CALLER	MOVES	NOTES
Fundraiser.contribute	Anyone	USDG → Resonance	Routed atomically; irreversible
Fundraiser.claim / settleEpochs	Anyone	GBX mint / none	Sequential, batched
SignalGBX.stake / unstake	Holder	GBX ↔ contract	Symmetric; unallocated only
Resonance signal ops (±, ±Many)	Signaler	None	Accounting only; symmetric
Resonance.notifyRevenue	Router only	USDG in	Binding-gated
distribute* / sync / index / payFundRevenue	Anyone	USDG out	Fixed destinations
addStrategy / killStrategy / addBribeReward	Timelocked owner	None	Governed, delayed; kill permanent
setResonanceRouter · setResonance ×3 · setMinter	Owner/deployer once	None	One-time; never reversible
Strategy.buy	Anyone	Payment in, lot out	Epoch, deadline, cap checks
BribeRouter.routePayment / payFundPayment	Strategy / anyone	Payment → Fund	100% liability
Bribe.notify / claims / payFundReward	Anyone	Reward tokens	Claims pay entitled account only
Bribe.addRewardToken / deposit / withdraw	Resonance only	None	Registration permanent, ≤ 8
Fund.redeem / burnGBX	Holder / anyone	Burn; assets out	Atomic; burns forever
LiquidityPosition.harvestFees / receiver hook	Anyone / fixed depositor	Fees out / NFT in	Principal fixed; NFT never leaves

"Timelocked owner" means reachable only through the Resonance owner after its public delay. Everything else is permissionless from block one. Nothing is pausable, and nothing takes a fee.

APPENDIX E

Security invariants

Statements that must hold after every valid transaction - each falsifiable by a test, and tested.

PLAIN ENGLISH	IDENTITY
Minting never exceeds the ceiling, ever	$\text{lifetimeMinted} \leq 10^9 \times 10^{18}$
Burns never reopen capacity	$\text{remaining} = \text{ceiling} - \text{lifetimeMinted}$ (burns absent)
Receipts are fully backed	$\text{sGBX.totalSupply} = \text{GBX held by SignalGBX}$
Signals are balance-bounded	$\text{accountSignalWeight}[a] \leq \text{sGBX.balanceOf}(a)$
Signal sums agree	$\Sigma_s \text{accountSignals}[a][s] = \text{accountSignalWeight}[a]; \Sigma_a = \text{strategySignalWeight}[s]; \Sigma_s \text{strategyWeight} = \text{totalSignalWeight}$
Rewards mirror signals	$\text{Bribe}(s).\text{balanceOf}(a) = \text{accountSignals}[a][s]; \text{totalSupply} = \text{strategySignalWeight}[s]$
Revenue is conserved	$\text{accounted} \times 10^{18} = \text{pendingScaled} + \text{indexedScaled} + \Sigma \text{remainders} + (\text{claimable} + \text{fundLiability}) \times 10^{18}$
Rewards are conserved	the analogous 8-term per-token identity in every Bribe
Payments are total	$\text{BribeRouter.accountedBalance} = \text{fundPaymentLiability}$; no payment enters reward accounting
Redemption is snapshotted and atomic	$\text{payout}_j = \text{floor}(\text{balance}_j \times \text{burn} / \text{supplyBefore})$; all-or-nothing
Exits move no tokens	<code>removeSignal</code> and <code>unstake</code> perform no USDG or reward-token transfer
Principal is permanent	position liquidity after any harvest = before; NFT custody has no exit path

HOW THEY WERE EXERCISED

27 invariant properties × 1,000 runs × depth 500 - 13.5 million randomized calls across 22 handler actions with zero reverts - plus Medusa's independent 101,840-call campaign over 62 property and assertion surfaces.

Explicitly *not* an invariant: exact temporal attribution of carry across a signal-supply change. That is open finding A-09, held visible by two deterministic proof-of-concept tests rather than papered over by the conservation identities - which all still hold.

APPENDIX F

Evidence and toolchain record

The exact tools behind Part XI's numbers, and where each recorded artifact lives in the repository.

TOOL	PIN	DISPOSITION
Foundry	1.7.1	All contract campaigns; re-run for this edition
Solidity	0.8.26, Cancun, opt. 10,000	Clean compile; two known bugs reviewed inapplicable
Hardhat / Node / pnpm	2.29.0 / 22.23.1 / 10.14.0	Parity, supply, workspace gates
Medusa	1.5.1	Passed: 101,840 calls, 62/62 surfaces
Echidna	2.3.2 pinned; 2.3.3 fallback	Blocked / invalid - release blocker
Mythril	0.24.8	Fails closed on Cancun opcodes - blocker
Slither / Aderyn / Semgrep / Solhint / Gitleaks	0.11.5 / 0.6.8 / 1.162.0 / 6.0.1 / 8.30.1	186 dispositioned; 0 Semgrep; 6 matches open
Mutation framework	none pinned	No current score - release blocker

COMMITTS

Baseline 395a0df; register candidate 54e3f2c; described head 5ad1ebc. The head differs from the candidate by the Fundraiser increment hardening and the ADR 0022 harvest redesign, both covered by the finalized campaign.

ENVIRONMENT CAVEATS FROM THIS PASS

Reproduction here used Foundry 1.7.1 and the pinned Node; the subgraph's Matchstick binary would not execute on this host (spec checks passed 4/4), and Docker-dependent gates remain blocked as recorded.

Primary records live under `packages/contracts/audit/`: FINDINGS, TEST-CAMPAIGN, STATIC-ANALYSIS, MUTATION-TESTING, FORMAL-CHECKS, UNISWAP-V4-REVIEW, FORK-VALIDATION, EXTERNAL_FUZZING, RESIDUAL-RISKS, RELEASE-CHECKLIST. This paper quotes those records; it does not replace them.

APPENDIX G

Residual risks and the deployment checklist

The recorded list of what stays true even if everything works - and what must be verified before deployment could be responsible.

Standing residual risks

Unpaid maintenance	Harvests, settlements, distributions, and burns wait for volunteers; realization timing is uncontrolled
Harvest-time allocation	Routed lumps follow the signal weights of that moment; signaling has no cooldown
A-05 by design	Expired auctions clear at zero and restart from the floor
A-09 open	Carry can cross signal-supply boundaries; material mainly for low-decimal tokens
Registry-free custody	Unsolicited and worthless tokens accumulate in the Fund indefinitely
Blocked tokens	A freezing token strands its own payouts (retryable, fixed-destination) but never anyone's exit
Donations	Direct transfers to any core contract are visible but have no recovery path

STATUS IN ONE LINE

Internal adversarial review completed with one open Medium finding; independent review required; deployment neither performed nor authorized.

Deployment verification checklist (condensed)

Independent audit passed and findings resolved · A-09 resolved or formally accepted · mutation campaign with reviewed survivors · pinned Echidna and Mythril runs · six secret-scan classifications · licensing and provenance cleared by counsel · signed manifest proving chain ID, bytecode, constructor arguments, timelock roles and delay, minter lock, all one-time bindings, PoolKey, ticks, token ID, and NFT custody · frontend read-only until the manifest passes. Every unchecked box is a reason not to deploy - and none were checked when this edition shipped.

APPENDIX H

Event guide

The observable record. Every meaningful state change emits; indexers and auditors need nothing else.

SURFACE	EVENTS AND MEANING
GBX supply	Minted, Burned, MinterSet: every supply change and the one-time handover
Mining	Contributed, EpochSettled, Claimed: who paid, what settled, what minted
Staking	Staked, Unstaked, ResonanceSet
Signals	SignalAdded, SignalRemoved - plus the Bribe's SignalWeightDeposited / Withdrawn mirror
Revenue	RevenueNotified, RevenueSynced, RevenueDistributed, FundRevenueAccrued, FundRevenuePaid
Governance	StrategyAdded, StrategyKilled, BribeRewardAdded, ResonanceRouterSet - the whole governed surface
Auctions	Purchased: buyer, receiver, epoch, lot, payment
Settlement	PaymentRouted, FundPaymentAccrued, FundPaymentPaid
Rewards	RewardAdded, Notified, Queued, StreamStarted (with exact rate and remainder), Paused, Resumed, Paid, FundRewardAccrued, FundRewardPaid
Fund	Redeemed (account, receiver, burn, token count), GBXBurned
Liquidity	PositionRecorded, FeesHarvested (principal, routed USDG, burned GBX)

The repository's subgraph indexes these into five entities with 36 handlers; auction fills are its one notable gap (readable directly from Strategy events). Presentation layers remain conveniences - the events are the record.

APPENDIX I

Glossary

Definitions as this paper uses them.

Basis point (bps)

One hundredth of one percent.

Blockchain

A public ledger maintained by many computers; finalized history cannot be quietly edited.

Bribe

The technical name of a Strategy's reward contract; an open incentive pot, capped at eight tokens. Plain prose: "Strategy reward".

Dutch auction (reverse)

A sale whose required payment falls with time; here the Strategy sells USDG and the falling number is the buyer's payment.

EIP-1153 / transient storage

Cancun-era scratch storage lasting one transaction; used for O(n) duplicate detection in redemption.

Epoch

A fixed period: one day for mining; a configured decay window for each auction.

ERC-20

The standard fungible-token interface (balances, transfers, approvals).

Exact-transfer check

Verifying both sides' balance deltas equal the requested amount; anything else reverts.

Fixed-point / WAD

Integers scaled by 10^{18} to represent fractions exactly.

Fund

The ownerless treasury holding acquired assets; exits are redemption and GBX burning only.

Fundraiser

The public mining contract: USDG in, scheduled GBX out, pro rata per epoch.

Gas

The execution fee every transaction pays the chain.

GBX

The protocol token: capped lifetime mint, permit and votes extensions, redeemable, burnable.

In-kind redemption

Receiving the underlying tokens themselves rather than a cash equivalent.

Killed Strategy

One permanently excluded from future revenue; existing signal remains removable.

Liquidity pool / LP position / tick / range

A standing two-token market; a deposit into it earning fees within a chosen price range delimited by ticks.

Multisig

A wallet requiring several signers per action.

Permit2 / EIP-2612

Signature-based token approvals; production code uses EIP-2612 on its own tokens and does not depend on Permit2.

Reentrancy

Re-entering a contract mid-execution via callback; guarded against on every token-moving entry point.

Resonance

The signal registry and revenue allocator; the only owned core contract.

ResonanceRouter

The single fixed entrance for USDG revenue.

Reward stream

A seven-day exact-rate distribution of a notified amount across signal weight.

sGBX / SignalGBX

The non-transferable 1:1 staking receipt measuring signal capacity.

Signal

An absolute amount of sGBX pointed at one Strategy; a delta-adjustable claim on future flow direction.

Smart contract

A program on the chain; immutable here by construction.

Stablecoin / USDG

An issuer-backed token targeting \$1; USDG (6 decimals) is this protocol's revenue asset.

Staking

Depositing GBX for sGBX at par; reversible for any unallocated amount.

Strategy

An immutable auction contract accumulating USDG toward one configured payment token.

Timelock

A contract executing approved actions only after a public delay; the intended Resonance owner.

Wallet

The keypair you act with; its security is yours alone.

APPENDIX J

References

Primary sources wherever they exist. Nothing security-critical in this paper rests on secondary commentary.

SOURCE	WHAT IT GROUNDS
Repository at 5ad1ebc...	Every contract behavior (packages/contracts/src/core)
packages/contracts/audit/	All evidence, findings, and blocked-gate statements
docs/whitepaper/FACT-CHECK.md	The claim-by-claim verification register behind this paper
packages/simulations/	Independently tested TS and Python arithmetic, cross-checked at build time
OpenZeppelin 5.6.1	ERC-20, Permit, Votes, ReentrancyGuard, TimelockController semantics
Uniswap v4 1.0.2 / 1.0.3 (pinned)	Pool, position, and fee-collection behavior
EIP-20, EIP-712, EIP-1153, EIP-2612	Token standard, typed signing, transient storage, permits
Solidity 0.8.26 release notes	Compiler semantics; the two reviewed known issues
Robinhood Chain reads (4663 @ 32,035,314)	EIP-1153 availability; documented v4 addresses
USDG issuer documentation	Stablecoin properties and issuer powers, as external assumptions
NOTICE; LEGAL-PROVENANCE-BLOCKER	Upstream lineage and the unresolved licensing record
ADRs 0013-0022	Design history, including ADR 0021 settlement and ADR 0022 harvesting

CITATION STYLE

The prose avoids per-sentence footnotes by design; every quantitative claim routes through the fact-check register, which cites file-level sources.

DISCLOSURE

Legal and risk disclosure

Plain language, complete, and binding on how this document may be read.

GumBall6900 is experimental software. This whitepaper explains it; it promises nothing. It is not an offer, solicitation, or recommendation of any asset or activity, and it is not legal, tax, investment, accounting, or financial advice. No one should contribute, stake, signal, trade, or redeem anything solely because of this document.

Nothing is guaranteed. GBX is not a stablecoin and is not represented as tracking the Fund's value. Public mining does not guarantee profit. Signaling does not guarantee rewards. Auctions may settle unfavorably, at zero, or not at all. Fund assets - including any tokenized-stock wrappers - may freeze, fail, become illiquid, lose value, or become legally restricted, and such wrappers do not confer the rights of directly registered shares. Supplemental reward tokens can fail independently. Smart-contract testing and internal review reduce risk; they do not eliminate it, and the contracts cannot be patched after deployment.

Law applies and varies. Securities, collective-investment, commodities, money-transmission, stablecoin, sanctions, tax, and consumer-protection regimes may each apply to some or all of this design, differently by jurisdiction. No legal review of this protocol has been performed; the repository's own licensing and provenance remain unresolved and are recorded as distribution blockers. Anyone considering interaction with any future deployment should obtain their own professional advice.

Status, restated. Not deployed. Not audited externally. One internal Medium finding open. Not authorized for user funds. The authors may abandon, change, or restart the project, and this document may become stale the day after it is built.

IF YOU REMEMBER ONE THING

This paper is a description of code, not an invitation to fund it. Treat every hopeful sentence in it as conditional on reviews that have not happened yet.

DOCUMENT BASIS

If this paper and deployed bytecode ever disagree, the bytecode wins.

This edition describes the production contracts at commit `5ad1ebc` of the GumBall6900 repository, reviewed internally against candidate `54e3f2c`. Canonical prose lives in `docs/WHITEPAPER.md`; this typeset edition is generated reproducibly from `docs/whitepaper/` by `pnpm docs:whitepaper`, which verifies protocol facts against contract constants and tested simulation fixtures, audits layout and contrast, and refuses to publish over a good PDF on any failure.

Charts are computed at build time; the worked example is computed by the same integer arithmetic the contracts apply; and the claim-by-claim record - including every place this paper's brief differed from the final implementation - is `docs/whitepaper/FACT-CHECK.md`.

BRAND AND TYPE

Brand palette: pink, blue, black, white. The display face Modak is vendored under the SIL Open Font License; body, heading, and mono faces resolve from the build machine and are embedded by the printer.

EDITION

v0.3 · 10 August 2026 · by Heesho

STATUS

Internal review only. Not audited. Not deployed. Not authorized for user funds.

DISCLOSURE

Explanatory document only. Not investment, legal, or tax advice.